Лабораторное занятие № 4: Модель обработки запросов сервером в Anylogic

Цели занятия: построить агентную модель для изучения процесса обработки запросов сервером в Anylogic.

Задание:

На основе лекционного материала и навыков, полученных на практических занятиях, в среде Anylogic построить агентную модель для изучения процесса обработки запросов сервером, решив следующие задачи:

- постановка задачи;
- создание диаграммы процесса;
- изменение свойств блоков модели, её настройка и запуск;
- создание анимации модели;
- сбор статистики использования ресурсов;
- уточнение модели согласно ёмкости входного буфера;
- сбор статистики по показателям обработки запросов;
- добавление параметров и элементов управления;
- добавление гистограмм;
- изменение времени обработки запросов сервером.

Постановка задачи

Сервер обрабатывает запросы, поступающие с автоматизированных рабочих мест с интервалами, распределенными по показательному закону со средним значением 2 мин. Время обработки сервером одного запроса распределено по экспоненциальному закону со средним значением 3 мин. Сервер имеет входной буфер ёмкостью 5 запросов.

Построить имитационную модель для определения математического ожидания времени и вероятности обработки запросов.

Сервер представляет собой однофазную систему массового обслуживания разомкнутого типа с ограниченной входной емкостью, то есть с отказами, и абсолютной надёжностью (рис. 1). На рис 1 приведены также объекты AnyLogic, которые будут использованы для создания диаграммы процесса.

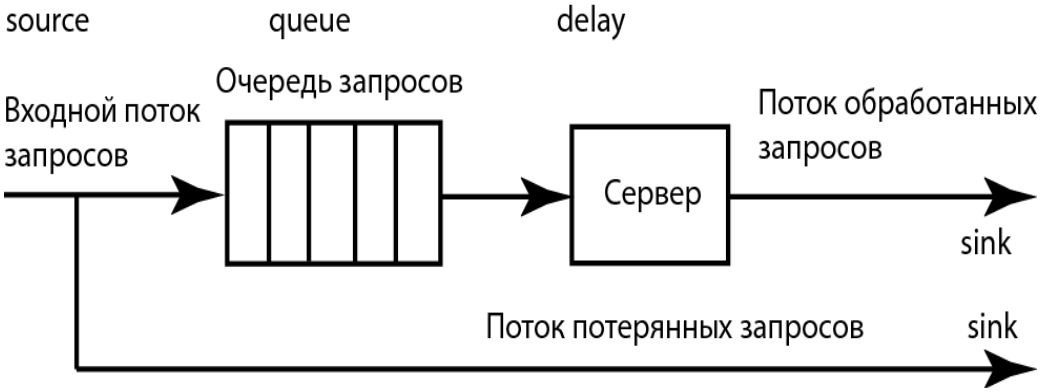

Рис. 1 - Сервер как система массового обслуживания

Создание диаграммы процесса

1. Выполните **Файл/Создать/Модель** на панели инструментов. Появится диалоговое окно **Новая модель** (рис. 2).

2. Задайте имя новой модели. В поле **Имя модели** введите **Server**.

3. Выберите каталог, в котором будут сохранены файлы модели.

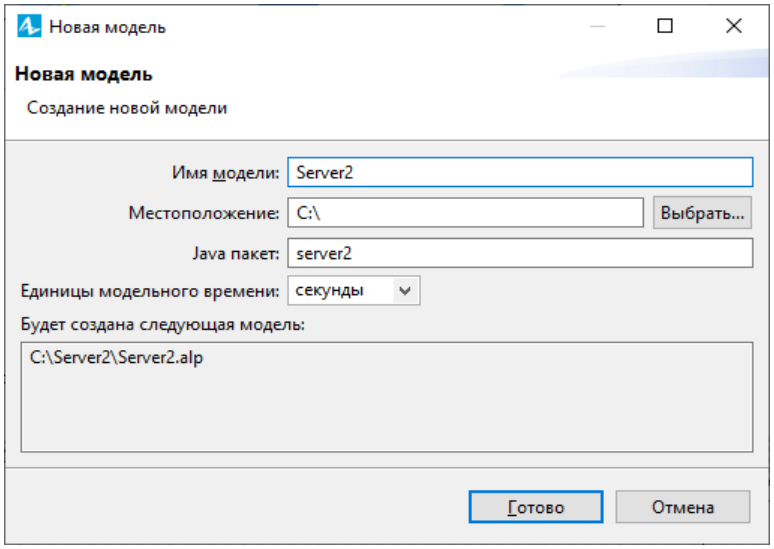

Рис. 2 - Диалоговое окно **Новая модель**

4. Щёлкните кнопку **Готово**. Откроется пользовательский интерфейс (рис. 3).

5. В левой части рабочей области находятся панель **Проекты** и панель **Палитра**. Панель **Проект** обеспечивает навигацию по элементам моделей, открытых в текущий момент времени. Модель организована иерархически. Она отображается в виде дерева. Сама модель образует верхний уровень дерева. Эксперименты, классы активных объектов и Java классы образуют следующий уровень. Элементы, входящие в состав активных объектов, вложены в соответствующую подветвь дерева класса активного объекта.

6. Панель **Палитра** (левый вертикальный столбец) содержит разделённые по категориям элементы, которые могут быть добавлены на графическую диаграмму типа агентов или эксперимента. На рис. 3 раскрыта палитра **Презентация**. Для того чтобы открыть нужную палитру, следует подвести курсор к иконке и щелкнуть мышью. Иконка становится светлой.

7. В правой части отображается панель **Свойства**. Панель **Свойства** используется для просмотра и изменения свойств выбранного в данный момент элемента (или элементов) модели.

8. В центре рабочей области AnyLogic размещён **графический редактор** диаграммы агента Main.

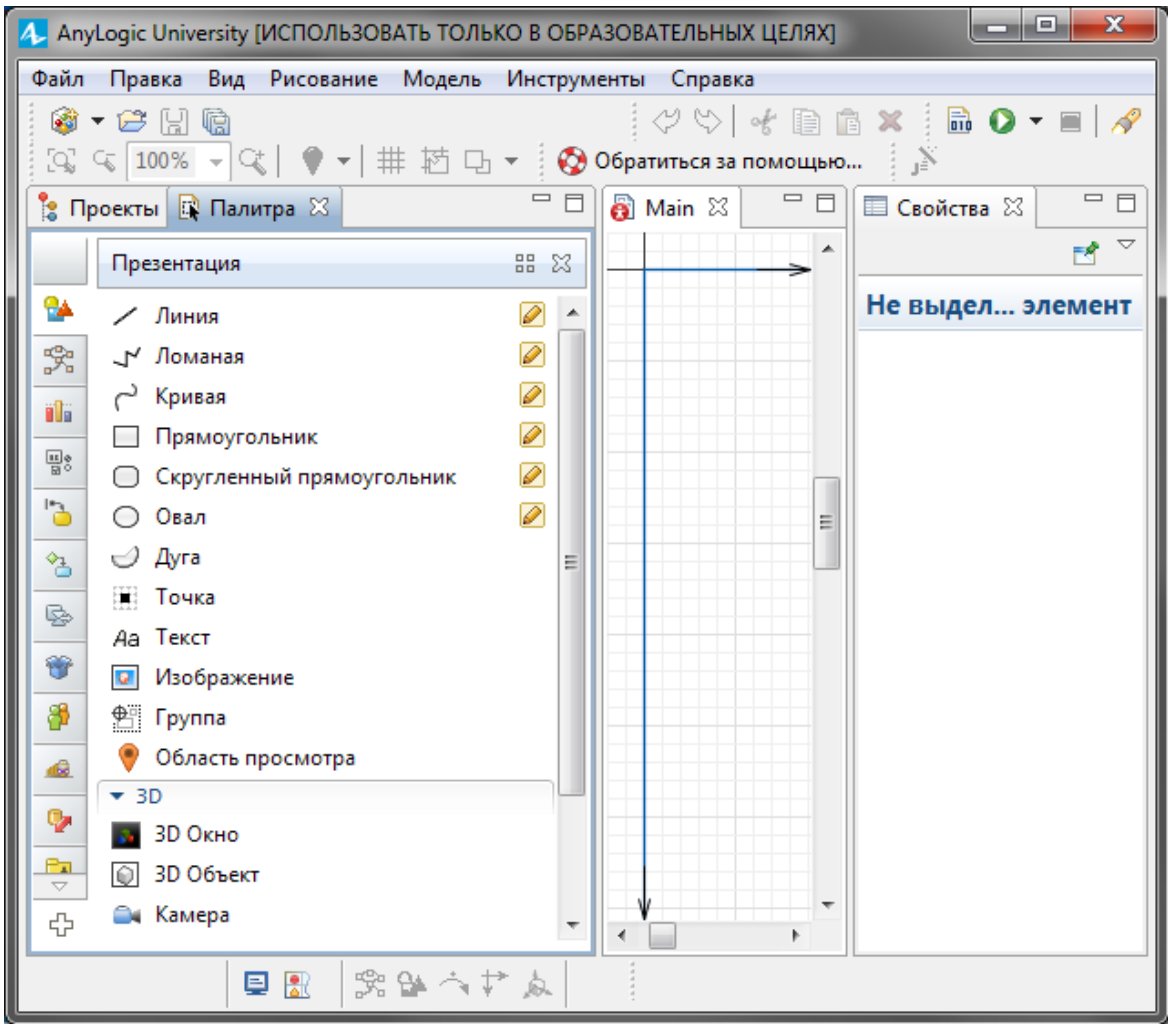

Рис. 3 - Пользовательский интерфейс

9. При работе не забывайте сохранять производимые изменения с помощью кнопки панели инструментов Сохранить.

10.Создайте диаграмму процесса. Для этого в Палитре выделите **Библиотеку моделирования процессов** (рис. 4). Из неё перетащите объекты на диаграмму и соедините, как показано на рис. 5. Для добавления объекта на диаграмму, надо щёлкнуть его мышью и, не отпуская её, перетащить в графический редактор.

11.При перетаскивании объектов вы можете видеть, как появляются соединительные линии между объектами. Обращайте внимание на то, чтобы эти линии, если необходимо, соединяли только порты, находящиеся с правой или левой стороны иконок. Если у вас не получится автоматическое соединение нужных объектов, дважды щёлкните начальный порт. Он станет зелёным. Протащите курсор к конечному порту. Он также станет зелёным. Для завершения процесса соединения дважды щёлкните конечный порт.

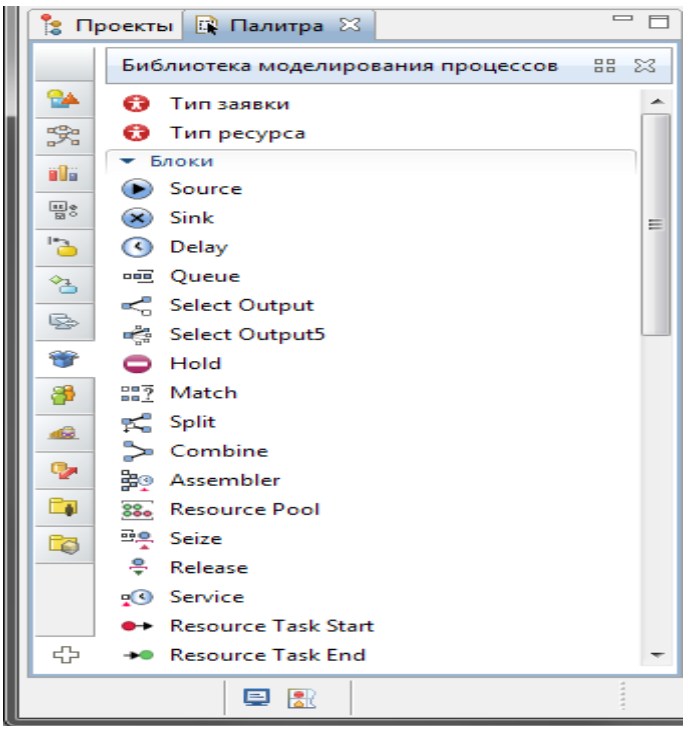

Рис. 4 - Библиотека моделирования процессов

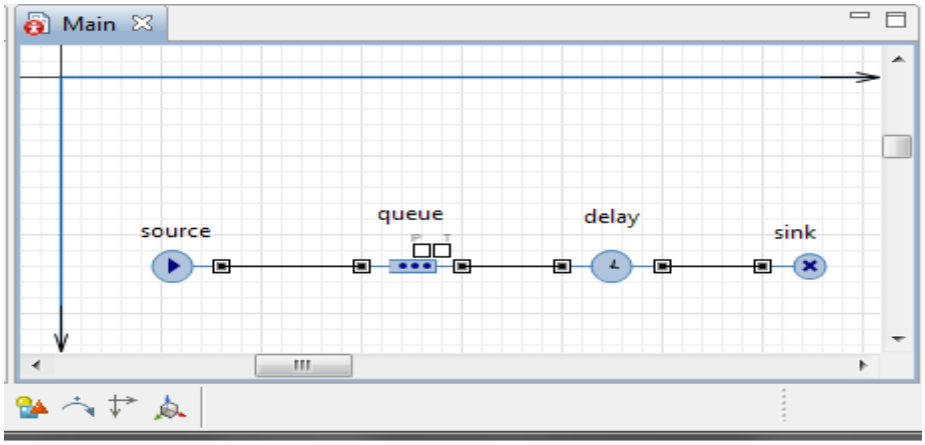

Рис. 5 - Диаграмма системы массового обслуживания

12. Объект **Source** генерирует заявки определенного типа. Обычно он используется в качестве начальной точки диаграммы процесса, формализующей поток заявок. В примере заявками будут запросы на обработку сервером, а объект **Source** будет моделировать их поступление.

13. Объект **Queue** моделирует очередь заявок, ожидающих приема объектами, следующими за данным в диаграмме процесса. В примере он будет моделировать очередь запросов, ожидающих освобождения сервера.

14. Объект **Delay** задерживает заявки на заданный период времени. Он представляет в нашей модели сервер, обрабатывающий запросы.

15. Объект **Sink** уничтожает поступившие заявки. Обычно он используется в качестве конечной точки потока заявок (и диаграммы процесса соответственно).

Изменение свойств блоков диаграммы процесса

В основе каждой дискретно-событийной модели лежит диаграмма процесса — последовательность соединенных между собой объектов (**Библиотеки моделирование процессов**), задающих последовательность операций, которые будут производиться над проходящими по диаграмме процесса заявками.

Диаграмма процесса в AnyLogic создаётся путем добавления объектов библиотеки из палитры на диаграмму класса активного объекта, соединения их портов и изменения значений свойств блоков в соответствии с требованиями модели.

Чтобы сделать созданную диаграмму модели (рис. 5) адекватной постановке задачи нужно изменить некоторые свойства объектов.

Свойства объекта (как и любого другого элемента AnyLogic) можно изменить в панели **Свойства**.

Обратите внимание, что панель **Свойства** является контекстно-зависимой. Она отображает свойства выделенного в текущий момент элемента. Поэтому для изменения свойств элемента нужно будет предварительно щелчком мыши выделить его в графическом редакторе или в панели **Проекты**.

Чтобы всегда была уверенность в том, что в текущий момент в рабочем пространстве выбран именно нужный элемент, и именно его свойства вы редактируете в панели **Свойства**, обращайте внимание на первую строку, показываемую в панели **Свойства** — в ней отображается имя выбранного в текущий момент времени элемента и его тип.

Согласно принятым стандартам, объекты в диаграмме процесса обычно располагаются цепочкой слева направо, представляя собой последовательную очередность операций, которые будут производиться над заявкой.

Первым объектом в диаграмме процесса является объект класса **Source**. Объект **source** генерирует заявки определенного типа. Заявки представляют собой объекты, которые производятся, обрабатываются, обслуживаются, или еще каким-нибудь образом подвергаются действию моделируемого процесса: это могут быть клиенты в системе обслуживания, детали в модели производства, транспортные средства в модели перевозок, документы в модели документооборота, сообщения в моделях систем связи и т.д. В примере заявками будут запросы на обработку данных, а объект **source** будет моделировать поступление запросов на сервер.

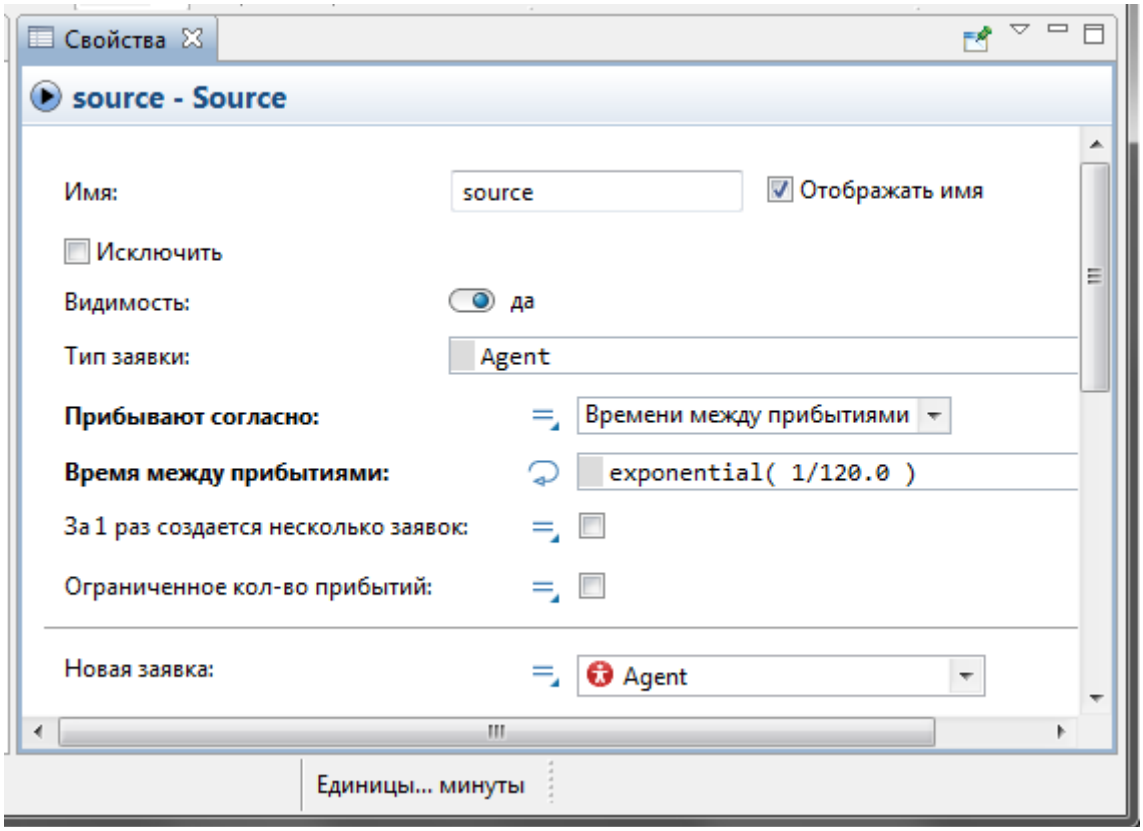

Рис. 6 - Свойства объекта **Source**

В примере объект создает заявки через временной интервал, распределенный по показательному (экспоненциальному) закону со средним значением 2 мин.

Установим среднее время поступления запросов и среднее время их обработки в секундах.

1. Выделите объект **Source**. В выпадающем списке **Прибывают согласно:** укажите, что запросы поступают согласно **Времени между прибытиями:** (рис. 6).

2. В поле **Время между прибытиями** появится запись exponential(1). Установите согласно постановке задачи среднее значение интервалов времени поступления запросов на сервер, изменив свойства объекта **Source**. Для этого вместо характеристики распределения 1 введите 1/120.0.

В языке программирования Java символ / означает целочисленное деление, т.е. если оба числа целые, то и результат будет целым. В примере отношение 1/120 было бы равно нулю. Для получения вещественного результата, необходимо, чтобы хотя бы одно из чисел было вещественным (double). Поэтому в качестве характеристики экспоненциального распределения (интенсивности поступления запросов) необходимо указать 1/120.0 или 1.0/120.

Следующий объект— **Queue**. Выделите его. Он моделирует очередь заявок, ожидающих приема объектами, следующими за данным объектом в

диаграмме процесса. В примере он будет, как уже отмечалось, моделировать очередь запросов, ждущих освобождения сервера.

Измените свойства объекта **Queue** (рис. 7).

1. Задайте длину очереди. Введите в поле **Вместимость:** 5. В очереди будут находиться не более 5 запросов.

2. Установите флажок **Включить сбор статистики**, чтобы включить сбор статистики для этого объекта. В этом случае по ходу моделирования будет собираться статистика по количеству запросов в очереди. Если же не установить этот флажок, то данная функциональность будет недоступна, поскольку по умолчанию она отключена для повышения скорости выполнения модели. Для вывода, например, средней длины очереди, нужно в модели предусмотреть Java код.

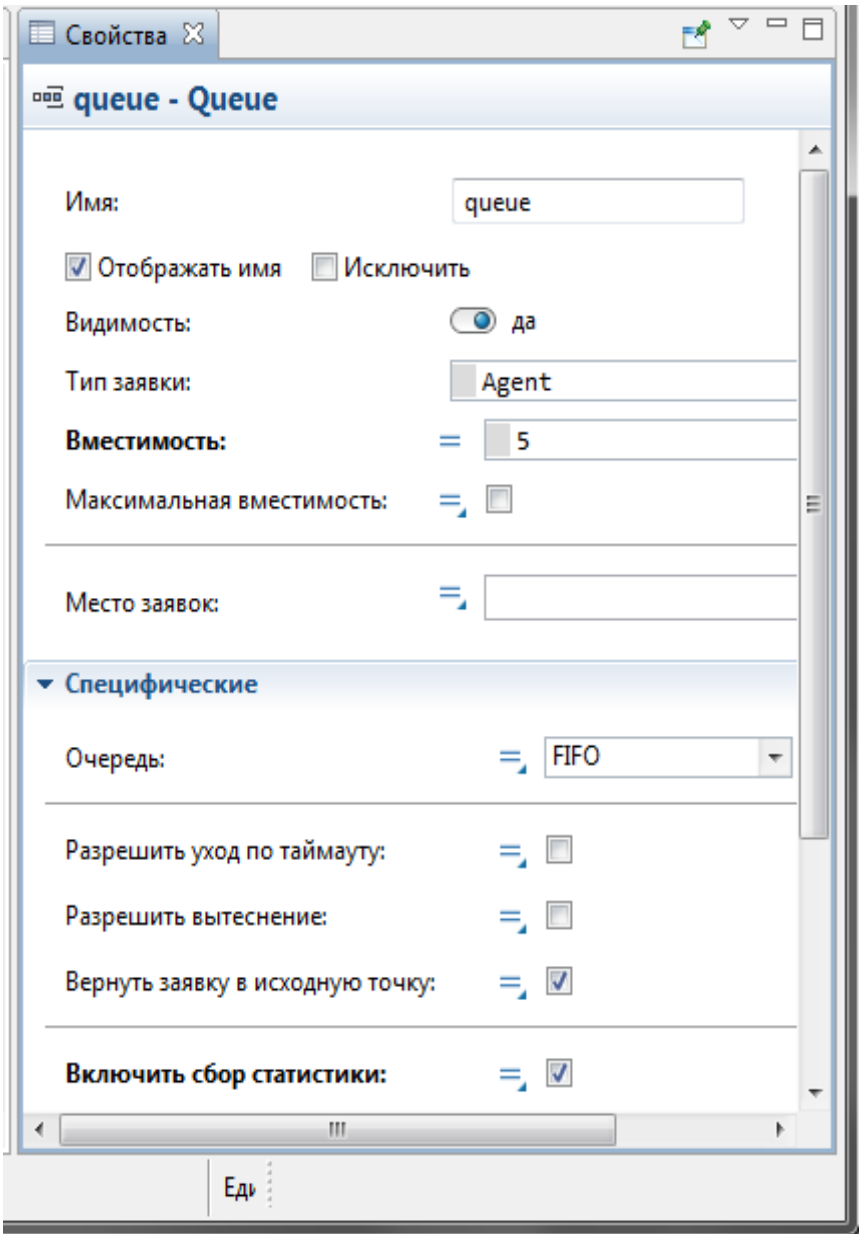

Рис. 7 - Свойства объекта **Queue**

Изменение свойств блоков диаграммы процесса

Следующим в диаграмме процесса расположен объект **delay**. Он задерживает заявки на заданный период времени, представляя в модели непосредственно сервер, на котором обрабатываются запросы.

Измените свойства объекта **delay** (рис. 8).

1. Обработка одного запроса занимает примерно 3 мин. Задайте время обслуживания, распределенное по экспоненциальному закону со средним значением 3 мин. Для этого введите в поле **Время задержки:** ***exponential(1/180.0)***.

2. Установите флажок **Включить сбор статистики**.

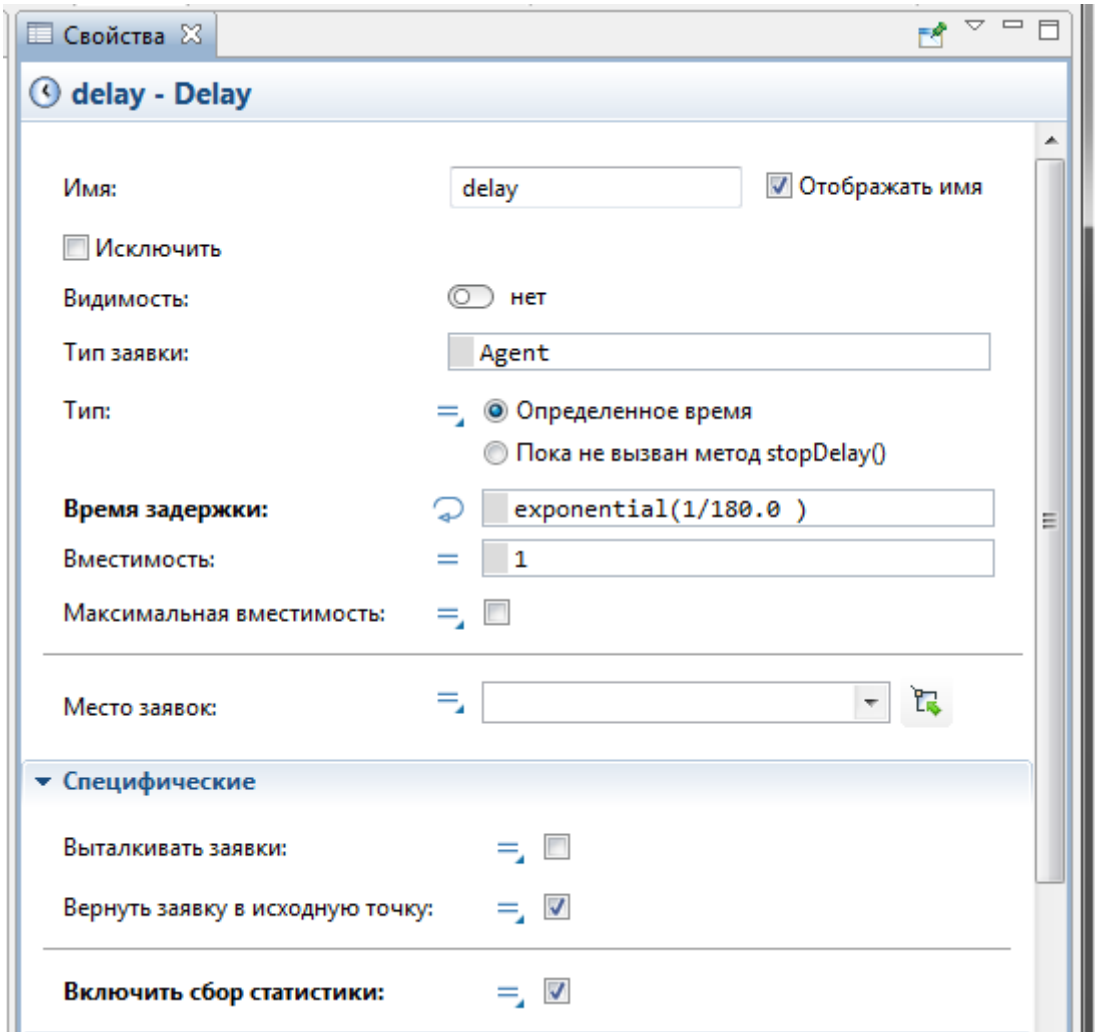

Рис. 8 - Свойства объекта **delay**

Последним в диаграмме нашей дискретно-событийной модели находится объект **sink**. Этот объект уничтожает поступившие заявки. Обычно он используется в качестве конечной точки потока заявок (и диаграммы процесса соответственно). В примере он выводит из модели обработанные сервером запросы.

Настройка запуска модели

Модель выполняется в соответствии с набором установок, задаваемым специальным элементом модели - экспериментом. Возможно создание несколько экспериментов с различными установками, изменение конфигурации модели.

В панели **Проект** эксперименты отображаются в нижней части дерева модели. Один эксперимент, названный **Simulation**, создается по умолчанию. Это простой эксперимент, позволяющий запускать модель с заданными значениями параметров, поддерживающий режимы виртуального и реального времени, анимацию и отладку модели.

Если нужно наблюдать поведение модели в течение длительного периода, то по умолчанию времени остановки нет. Обработку запросов сервером в примере планируем исследовать в течение одного часа, т.е. 3600 с.

1. В панели **Проект** выделите эксперимент **Simulation:Main**.
2. Щелчком раскройте вкладку **Модельное время**.
3. Установите **Виртуальное время (максимальная скорость)** (рис. 9).
4. В поле **Остановить:** выберите из списка **В заданное время**.
5. В поле **Конечное время:** установите **3600**.
6. Раскройте вкладку **Случайность**.
7. Выберите опцию **Фиксированное начальное число (воспроизводимые прогоны)**.
8. В поле **Начальное число:** установите **9**.
9. В панели **Проект**, выделите **Server** (рис. 10).
10. Из выпадающего списка **Единицы модельного времени:** выберите секунды.

Настройка запуска модели

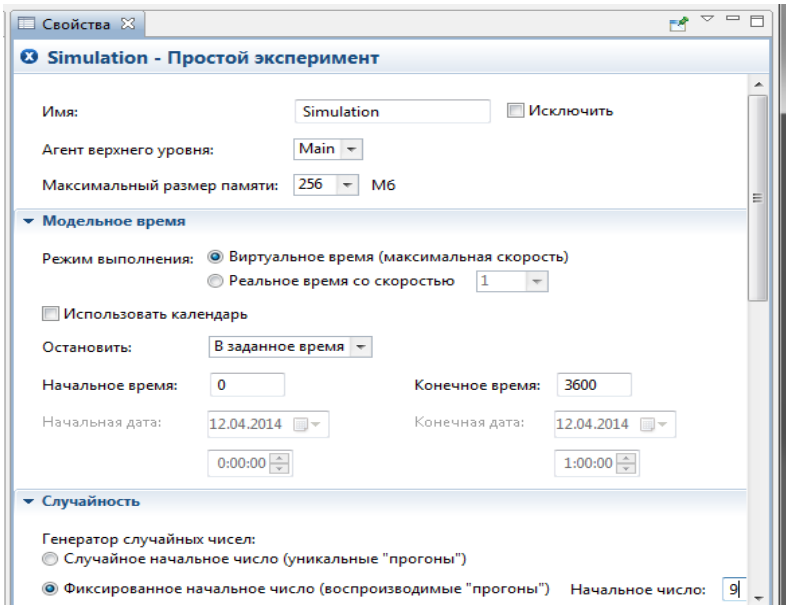

Рис. 9 - Установка свойств эксперимента

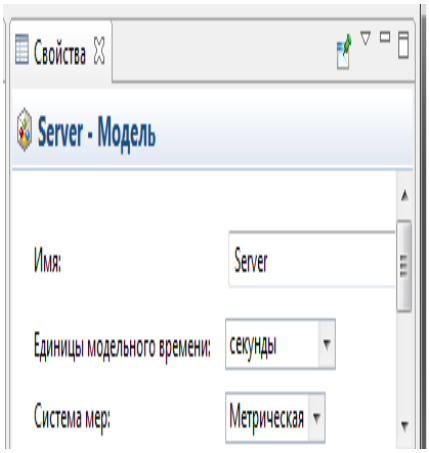

Рис. 10 - Установка модельного времени

Запуск модели

Постройте модель с помощью кнопки панели инструментов (F7) **Построить модель** (при этом в рабочей области AnyLogic должен быть выбран какой-то элемент именно этой модели). Если в модели есть какие-нибудь ошибки, то построение не будет завершено, и в панель **Ошибки** будет выведена информация об ошибках, обнаруженных в модели. Двойным щелчком мыши по ошибке в этом списке вы можете перейти к предполагаемому месту ошибки, чтобы исправить её. При этом откроется соответствующее место ошибки.

После исправления ошибок и построения модели, запустите её:

1. Щёлкните мышью кнопку панели инструментов **Запустить** (или нажмите F5) и выберите из открывшегося списка эксперимент, который вы хотите запустить. Эксперимент этой модели будет называться **Server/Simulation**.

2. В дальнейшем нажатием кнопки **Запустить** (или кнопки F5) будет запускаться тот эксперимент, который запускался в последний раз. Чтобы выбрать другой эксперимент, будет нужно щелкнуть мышью по стрелке, находящейся в правой части кнопки **Запустить**, и выбрать нужный эксперимент из открывшегося списка (или щелкнуть правой кнопкой мыши по этому эксперименту в панели **Проект** и выбрать **Запустить** из контекстного меню).

3. После запуска модели станет видимым окно презентации этой модели (рис. 11). В нем будет отображена презентация запущенного эксперимента. AnyLogic автоматически помещает на презентацию каждого простого эксперимента заголовок и кнопку, позволяющую запустить модель и перейти на презентацию, нарисованную для главного класса активного объекта этого эксперимента (**Main**).

4. Щёлкните данную кнопку. Этим щелчком запустится модель и осуществится переход к презентации корневого класса активного объекта запущенного эксперимента. Для каждой модели, созданной в **Библиотеке моделирования процессов**, автоматически создается блок-схема с наглядной визуализацией процесса, с помощью которой можно изучать текущее состояние модели, например, длину очереди, количество обработанных запросов и так далее (рис. 12).

Запуск модели

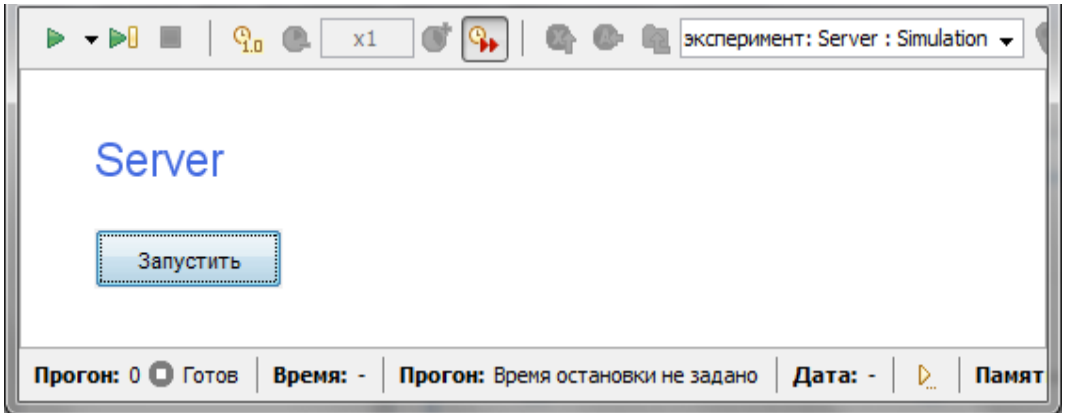

Рис. 11 - Окно презентации модели

5. Для каждого объекта определены правила, при каких условиях принимать заявки. Некоторые объекты задерживают заявки внутри себя, некоторые - нет. Для объектов также определены правила: может ли заявка, которая должна покинуть объект, ожидать на выходе, если следующий объект не готов её принять. Если заявка должна покинуть объект, а следующий объект не готов её принять, и заявка не может ждать, то модель останавливается с ошибкой (рис. 12). Ошибка означает, что запрос не может покинуть объект **Source** и войти в блок **Queue**, так как его ёмкость, равная 5, заполнена. Также выдаётся сообщение о логической ошибке в модели (рис. 13)

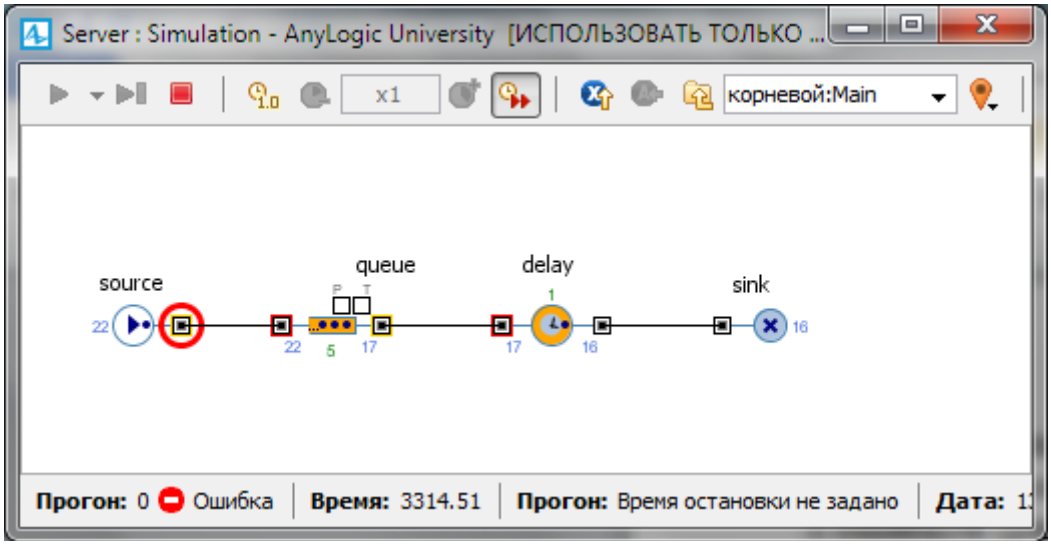

Рис. 12 - Модель остановилась с ошибкой

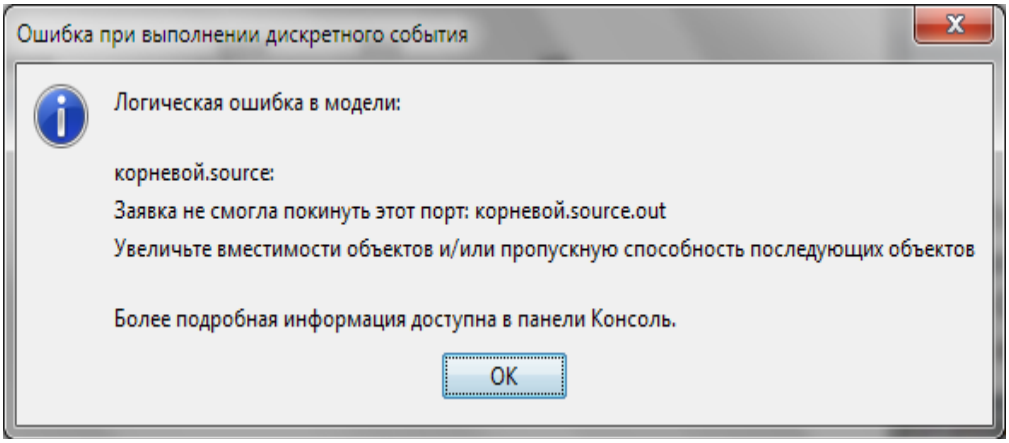

Рис. 13 - Сообщение о логической ошибке в модели

6. Нажмите OK. Далее измените свойства объекта **Queue**, т. е. увеличьте длину очереди (см. рис. 7). Для этого введите в поле **Вместимость** 15. Можете убедиться, что при увеличении ёмкости в пределах 6 - 14 модель по-прежнему останавливается с этой же ошибкой. Момент появления ошибки зависит от длительности времени моделирования.

7. Снова запустите модель.

8. Можно следить за состоянием любого объекта диаграммы процесса во время выполнения модели с помощью окна инспекта этого объекта. Чтобы открыть окно инспекта, щёлкните мышью по значку нужного блока. Окно инспекта, подведя курсор, можно перемещать в нужное место. Также, подведя курсор к правому нижнему углу окна инспекта, можно при необходимости изменять его размеры.

9. В окне инспекта будет отображена базовая информация по выделенному объекту: например, для объекта **Queue** будут отображены вместимость очереди, количество заявок, прошедшее через каждый порт объекта и т. д. Такая же информация содержится в инспекте и для объекта **delay** (рис. 14).

10. Для остановки выполнения модели щёлкните мышью кнопку **Прекратить выполнение** панели управления окна презентации.

11. Для предотвращения остановок модели по ранее указанной ошибке (недостаточной ёмкости объекта **queue)** была увеличена ёмкость объекта **queue**. Однако можно было бы изменять среднее время имитации поступления запросов объектом **source** и среднее время обработки запросов сервером, т. е. среднее время задержки объекта **delay**, оставляя неизменной длину очереди и добиваясь безошибочной работы модели. Конечно, при изменении свойств объектов модели нужно обязательно исходить из целей её построения.

Запуск модели

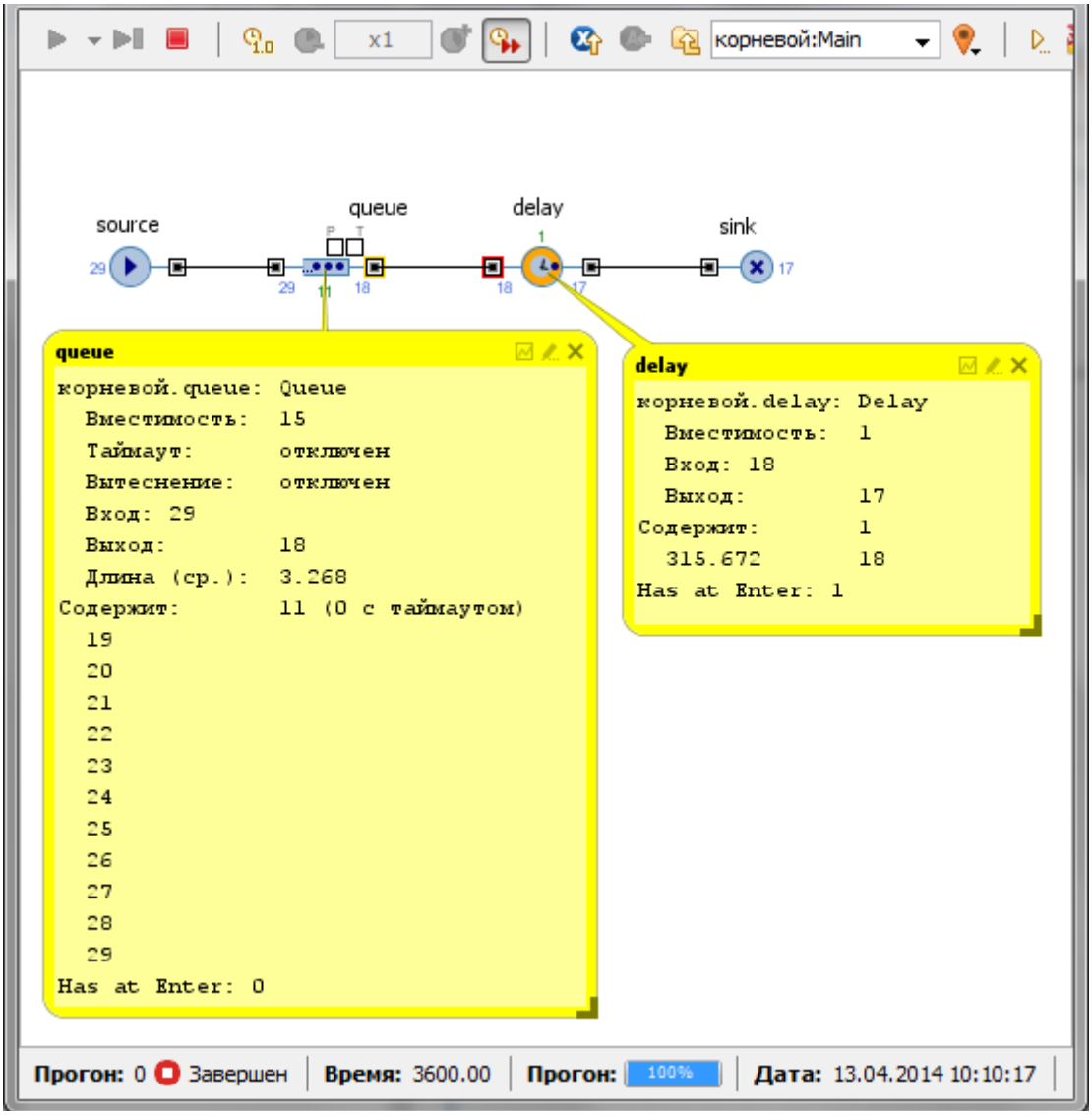

Рис. 14 - Окно инспекта

Создание анимации модели

Можно было бы наблюдать, анализировать и интерпретировать работу запущенной модели с помощью визуализированной диаграммы процесса (см. рис. 12, 14).

Однако удобнее в ряде случаев иметь более наглядную визуализацию с помощью анимации. Создадим визуализированный процесс поступления запросов на сервер и обработки запросов сервером.

Так как в данном случае не интересует конкретное расположение объектов в пространстве, то можно просто добавить схематическую анимацию интересующих объектов - сервер и очередь запросов к нему.

Анимация модели рисуется в той же диаграмме (в графическом редакторе), в которой задается и диаграмма моделируемого процесса.

Нарисуйте прямоугольный узел, который будет обозначать на анимации сервер.

1. Откройте палитру **Разметка пространства** (рис. 15). Чтобы открыть какую-либо палитру, нужно щелчком из **Проекты** перейти в **Палитра** и щёлкнуть по иконке этой палитры.

2. Палитра **Разметка пространства** (рис. 15) содержит в качестве элементов различные примитивные фигуры, используемые для рисования презентаций моделей. Это путь, прямоугольный узел, многоугольный узел, точечный узел, аттрактор, стеллаж, масштаб.

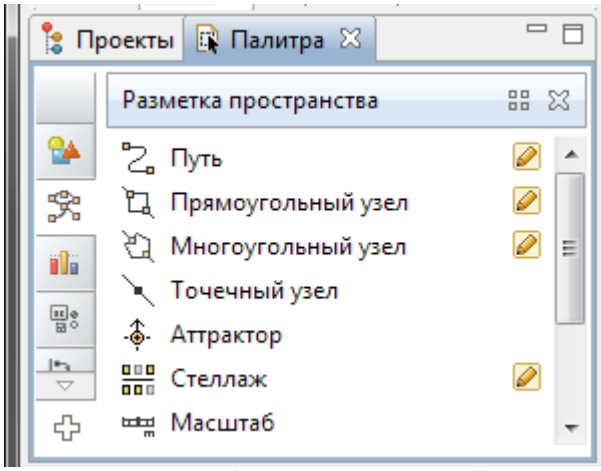

Рис. 15 - Палитра **Разметка пространства**

3. Выделите элемент **Прямоугольный узел** и перетащите его на диаграмму класса активного объекта. Поместите элемент **Прямоугольный узел** так, как показано на рис. 1.16.

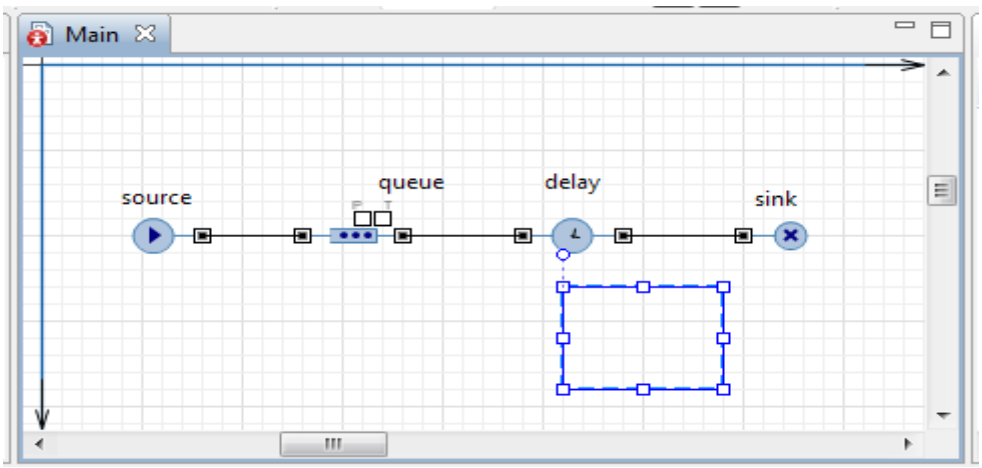

Рис. 16 - Элемент **Прямоугольный узел** на диаграмме

4. Можно сделать, чтобы цвет этого прямоугольного узла менятлся в зависимости от того, обрабатывает ли сервер в данный момент времени запрос или нет.

5. Для этого выделите нарисованную фигуру на диаграмме. Перейдите на страницу **Внешний вид** панели свойств (рис. 17).

Если нужно, чтобы по ходу моделирования то или иное свойство фигуры меняло своё значение в зависимости от каких-то условий, то можно ввести в поле соответствующего динамического свойства выражение, которое будет постоянно вычисляться заново при выполнении модели.

Возвращаемый результат вычисления будет присваиваться текущему значению этого свойства. Поскольку нужно, чтобы во время моделирования менялся цвет нашей фигуры, поэтому щёлкните в поле **Цвет заливки:** по стрелке, выберите **Динамическое значение** и введите там следующую строку: **delay.size()>0?red:green**

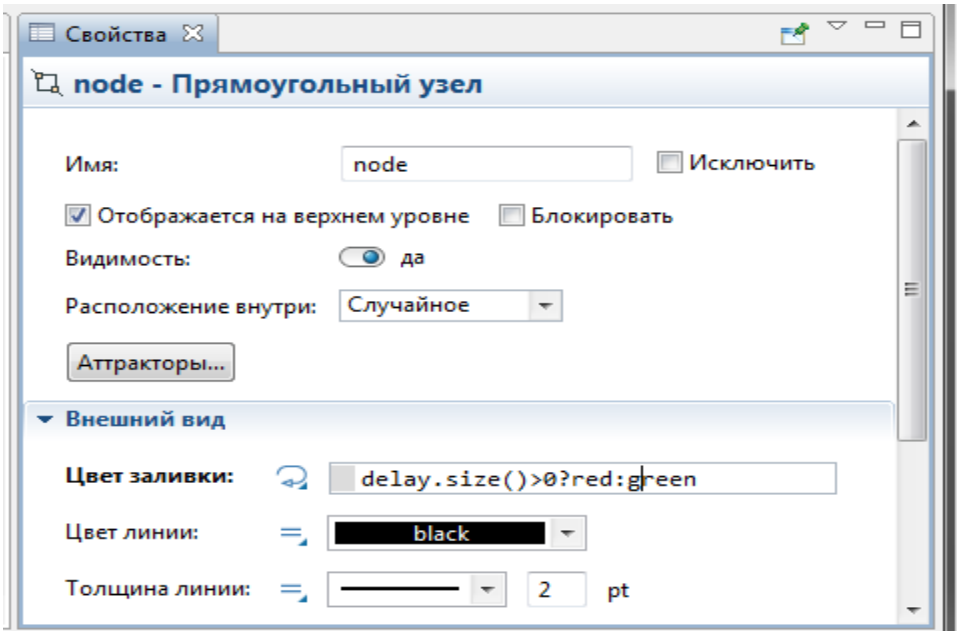

Рис. 17 - Установлено динамическое значение цвета заливки

Функция **size()** возвращает число запросов, обслуживаемых в данный момент времени. Если сервер занят, то цвет кружка будет красным, в противном случае — зелёным.

6. Нарисуйте путь, который будет обозначать на анимации очередь к серверу (рис. 18). Чтобы нарисовать путь, сделайте двойной щелчок мышью по элементу **Путь** палитры **Разметка пространства**, чтобы перейти в *режим рисования*. Теперь можно рисовать путь точка за точкой, последовательно щелкая мышью в тех точках диаграммы, куда нужно поместить вершины пути. Чтобы завершить рисование, добавьте последнюю точку пути двойным щелчком мыши.

Очень важно, какая точка пути создается первой. Заявки будут располагаться вдоль нарисованного вами пути в направлении от конечной точки к начальной точке. Поэтому обязательно начните рисование пути слева и поместите рядом с сервером конечную точку пути, которая будет соответствовать в этом случае началу очереди.

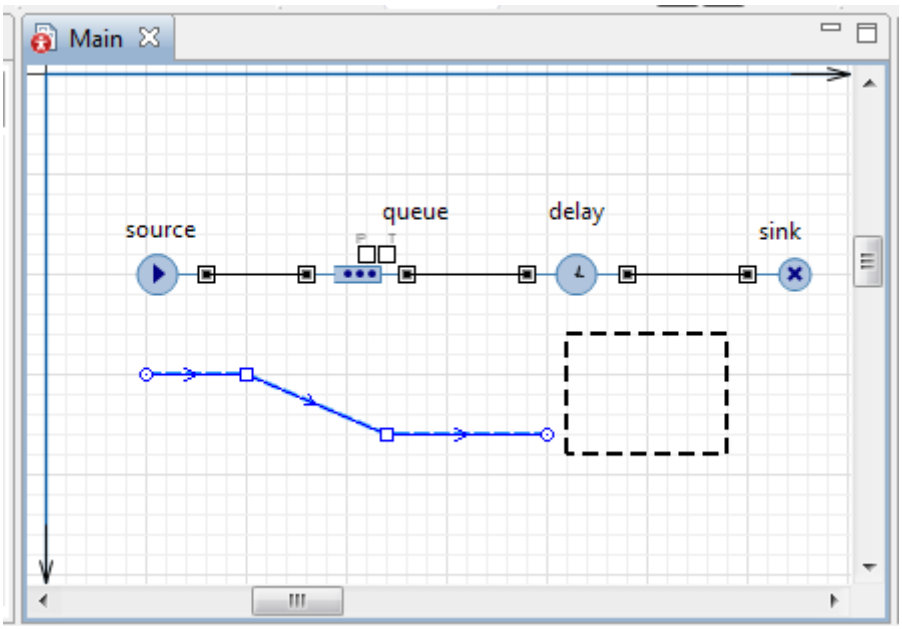

Рис. 18 - Путь на диаграмме процесса

7. Теперь нужно задать созданные анимационные объекты в качестве анимационных фигур объектов диаграммы процесса. Задайте путь в качестве фигуры анимации очереди. Выделите объект **queue**. На странице свойств объекта **queue** в поле **Место заявок:** выберите из выпадающего списка **path** (рис. 19).

8. Задайте прямоугольный узел в качестве фигуры анимации сервера. Выделите объект **delay**. Введите в поле **Место заявок:** из выпадающего списка имя нашего прямоугольного узла: **node** (рис. 20).

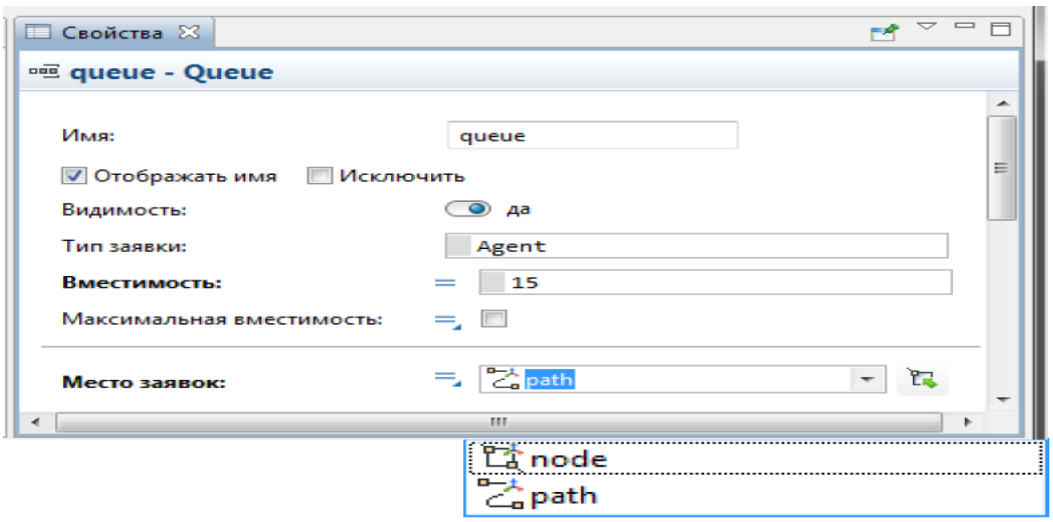

Рис. 19 - Задание пути в качестве фигуры анимации очереди

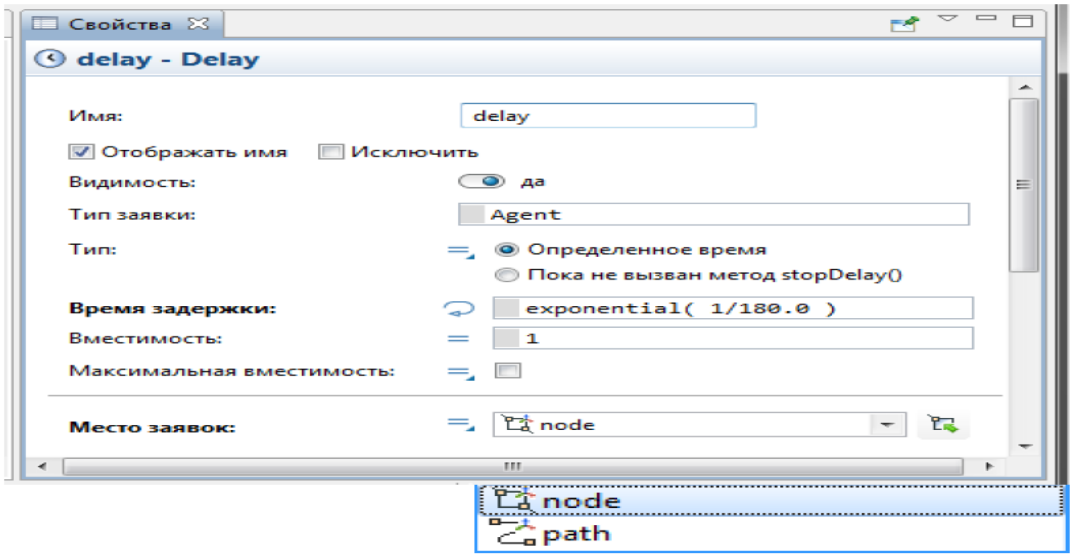

Рис. 20 - Задание прямоугольного узла в качестве фигуры анимации сервера

9. Запустите модель. У модели теперь есть простейшая анимация— сервер и очередь запросов к нему (рис. 21). Цвет фигуры сервера будет меняться в зависимости от того, обрабатывается ли запрос в данный момент времени или нет.

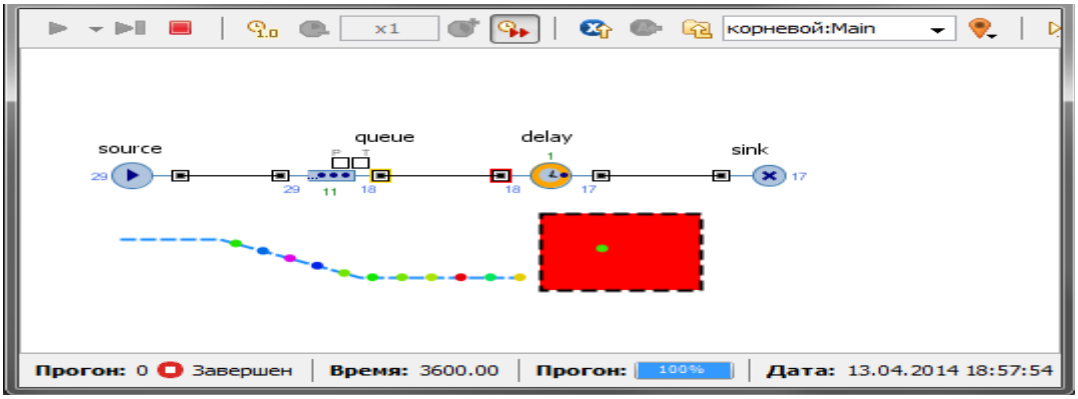

Рис. 21 - Анимация модели

Сбор статистики использования ресурсов

AnyLogic предоставляет пользователю удобные средства для сбора статистики по работе блоков диаграммы процесса. Объекты **Enterprise Library** самостоятельно производят сбор основной статистики.

Добавьте диаграмму для отображения среднего коэффициента использования сервера:

1. Откройте палитру **Статистика**. Эта палитра содержит элементы сбора данных и статистики, а также диаграммы для визуализации данных и результатов моделирования.

2. Перетащите элемент **Столбиковая диаграмма** из палитры **Статистика** на диаграмму класса и измените ее размер, как показано на рис. 22.

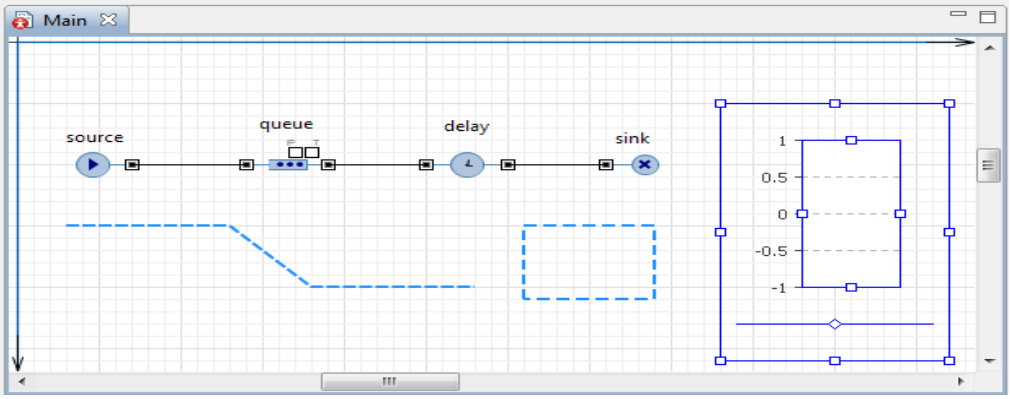

Рис. 22 - Элемент **Столбиковая диаграмма** на диаграмме класса

3. Перейдите на панель **Свойства**. Щёлкните кнопку **Добавить элемент данных**. После щелчка появится секция свойств того элемента данных (**chart – Столбиковая диаграмма**), который будет отображаться на этой диаграмме (рис. 23).

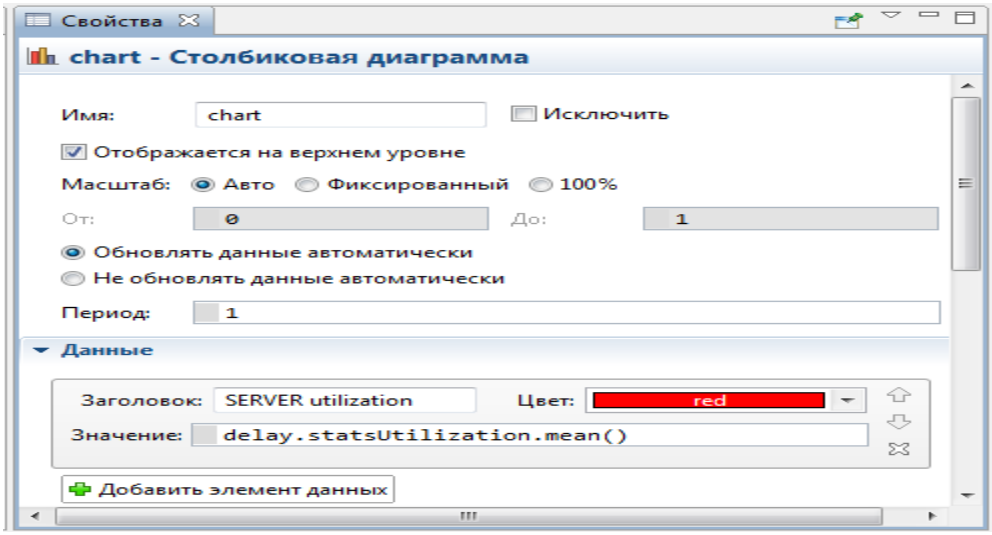

Рис. 23 - Страница **Свойства**

4. Измените **Заголовок** на **SERVER utilization**.

5. Введите **delay.statsUtilization.mean()** в поле **Значение**. У каждого объекта **delay** есть встроенный набор данных **statsUtilization**, занимающийся сбором статистики использования этого объекта. Функция **mean()** возвращает среднее из всех измеренных этим набором данных значений. Возможно использовать и другие методы сбора статистики, такие, как **min()** или **max()**. Полный список методов можно найти на странице документации этого класса набора данных: **StatisticsContinuous** (на английском языке).

6. Щёлкните **Внешний вид** (рис. 24). Установите свойства: направление столбцов, цвета фона, границ, меток, сетки, положение подписей у столбцов.

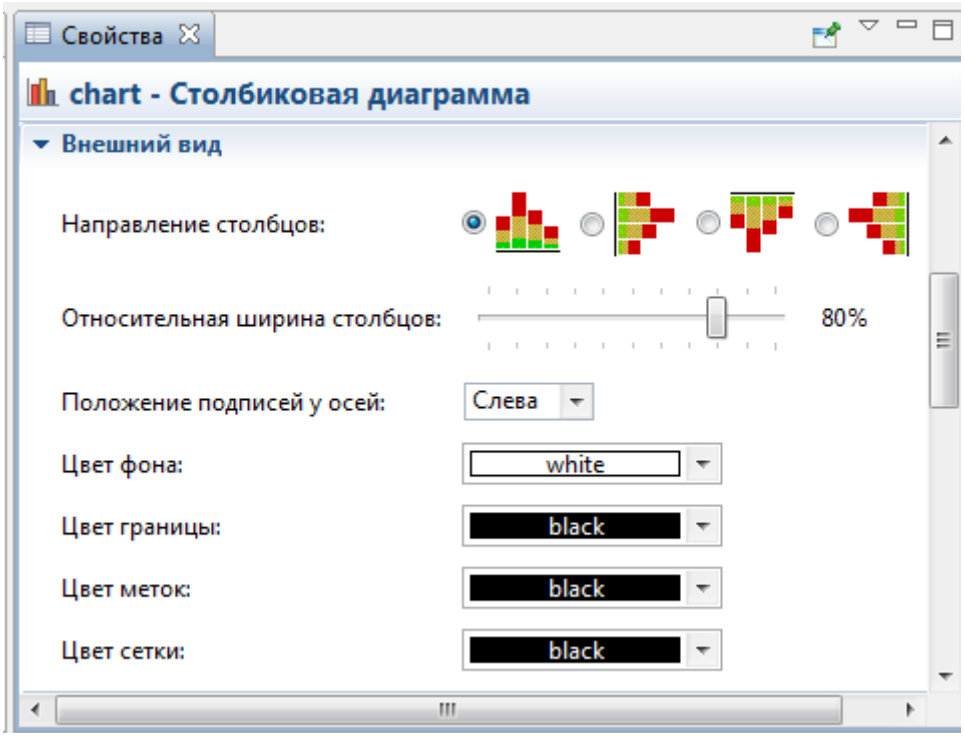

Рис. 24 - Вкладка **Внешний вид**

7. Раскройте щелчками страницы (вкладки) **Местоположение и размер**, **Легенда**, **Область диаграммы** (рис. 25). Установите свойства, чтобы изменить расположение легенды относительно диаграммы (чтобы она отображалась внизу), размер диаграммы, высоту, ширину, координаты размещения на диаграмме, цвета текста, границы.

8. Аналогичным образом добавьте еще одну столбиковую диаграмму для отображения средней длины очереди.

9. На панели **Свойства** щёлкните **Добавить элемент данных**. После щелчка появится страница **Данные** свойств элемента данных (**chart1 – Столбиковая диаграмма**), который также будет отображаться на этой диаграмме (рис. 26).

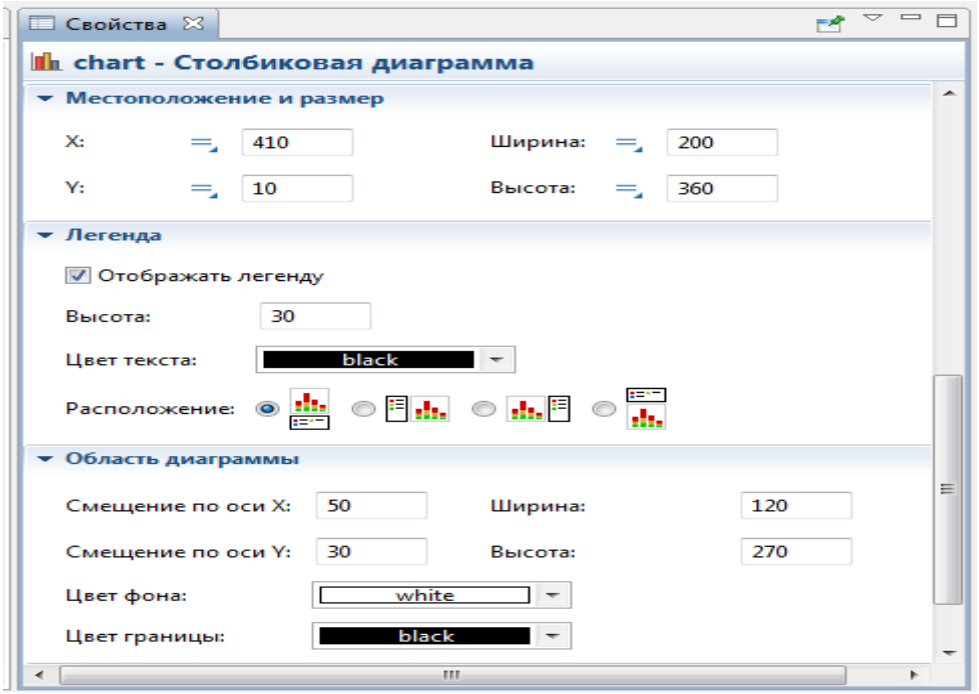

Рис. 25 - Вкладки **Местоположение и размер**, **Легенда**, **Область диаграммы**

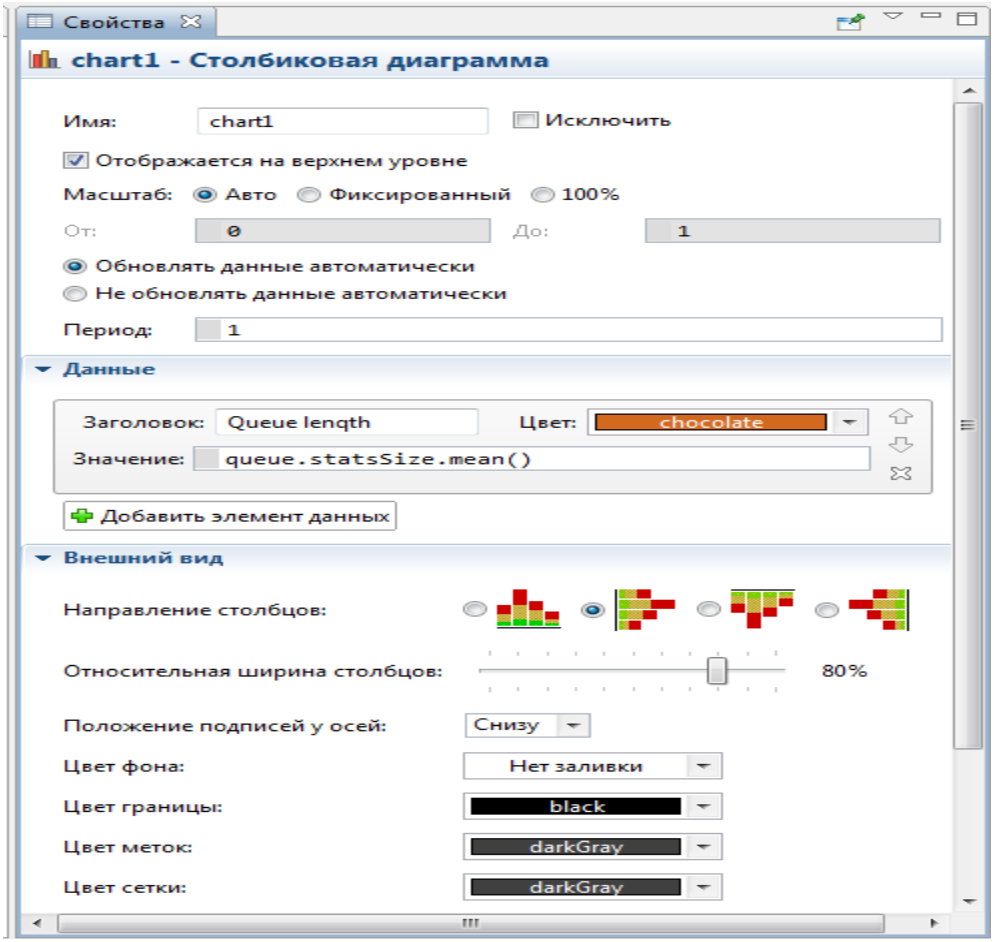

Рис. 26 - Страницы **Данные**, **Внешний вид** панели **Свойства**

10. **Заголовок:** и **Значение:** измените так, как показано на рис. 26. В поле **Заголовок:** введите **Queue lenqth**, а в поле **Значение:** введите **queue.statsSize.mean()**.

11. На страницах **Внешний вид**, **Местоположение и размер**, **Легенда**, **Область диаграммы** установите свойства самостоятельно. Столбцы диаграммы должны размещаться горизонтально.

12. В поле **Значение:** queue— это имя объекта **queue**. У каждого объекта **queue**, как и объекта **delay**, также есть встроенный набор данных statsSize, занимающийся сбором статистики использования этого объекта. Функция **mean()** также возвращает среднее из всех измеренных этим набором данных значений.

13. На странице **Внешний вид** панели **Свойства** выберите в секции свойств **Направление столбцов** вторую опцию (рис. 26), чтобы столбцы во второй столбиковой диаграмме, расположенной горизонтально, росли вправо (рис.27).

14. Запустите модель с двумя столбиковыми диаграммами, установив модельное время 3600 единиц, и понаблюдайте за её работой (рис. 28).

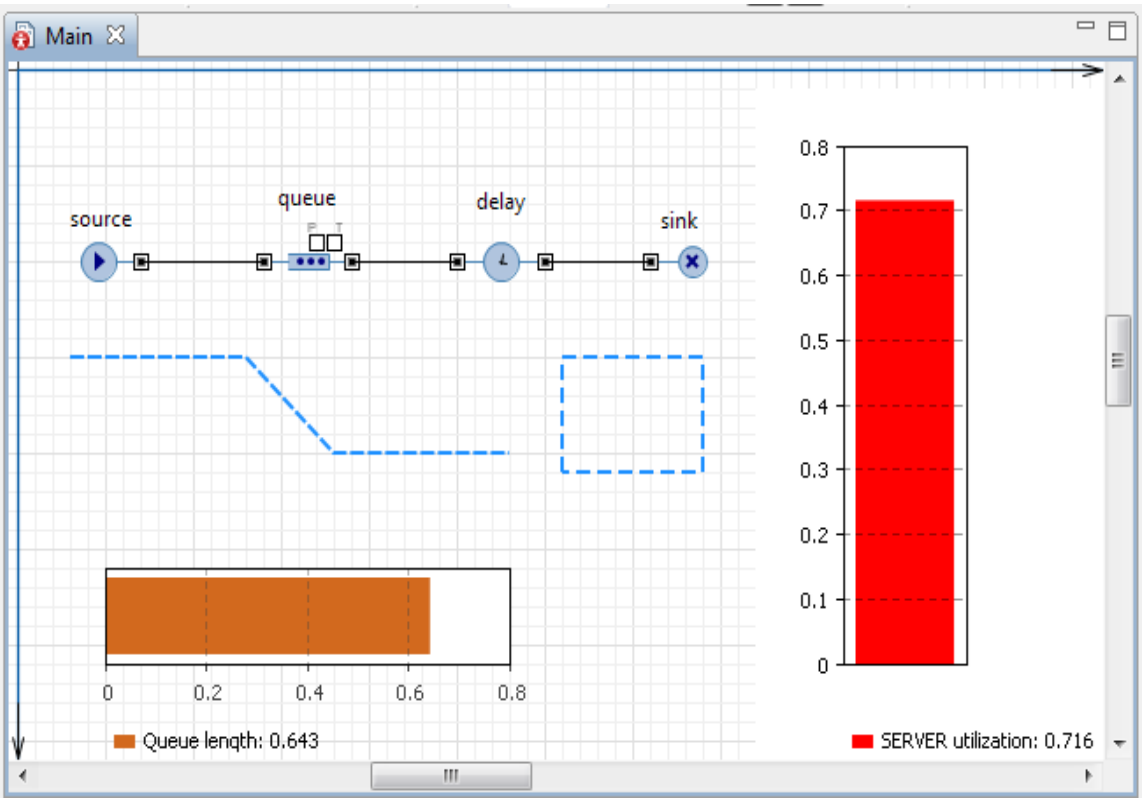

Рис. 27 - Добавлены две столбиковые диаграммы

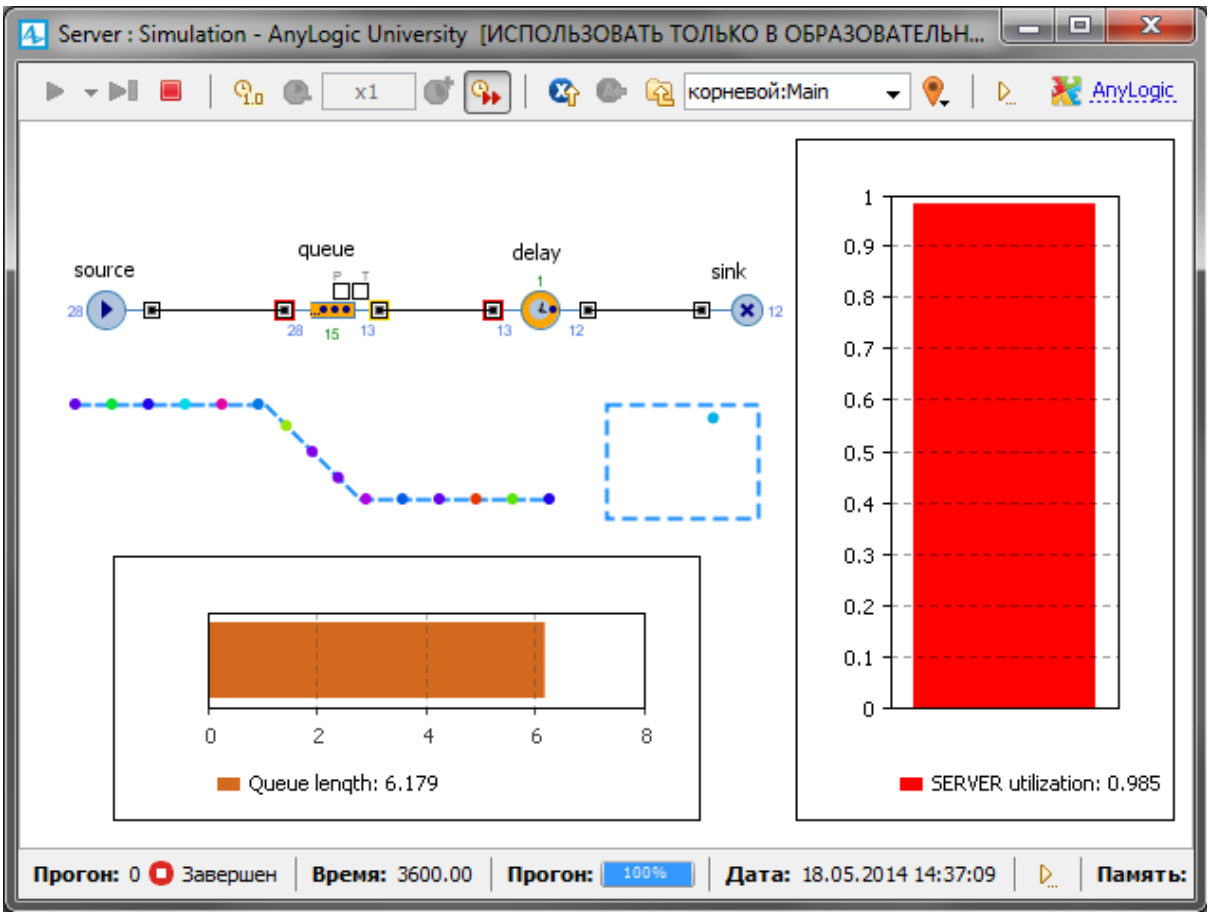

Рис. 28 - Наблюдение за моделью с двумя столбиковыми диаграммами

Уточнение модели согласно ёмкости входного буфера

На рис. 28 (снимок сделан по окончании времени моделирования) видно, что длина очереди равна 14 запросам при установленной максимальной длине 15. Но ведь в постановке задачи ёмкость буфера была определена в 5 запросов. Не удалось до этого построить модель с такой ёмкостью из-за ошибки (см. рис. 22) — невозможности очередного запроса покинуть блок **source**, так как длина очереди уже была равна 5 запросам. Пришлось во избежание этой ошибки увеличить ёмкость буфера до 15 запросов.

А возможно ли выполнить данное условие постановки задачи средствами AnyLogic? Да, причем, различными способами. Уточним модель согласно постановке задачи одним из этих способов.

Объект queue моделирует очередь заявок, ожидающих приёма объектами, следующими за ним в потоковой диаграмме, или же моделирует хранилище заявок общего назначения. При необходи-мости вы можете задать максимальное время ожидания заявки в очереди. Вы также можете с помощью написанной вами програм-мы извлекать заявки из любых позиций в очереди.

Заявка может покинуть объект **queue** различными способами:

- обычным способом через порт ***out***, когда объект, следующий в блок-схеме за этим объектом, готов принять заявку;
- через порт ***outTimeout***, если заявка проведет в очереди заданное количество времени (если включен режим таймаута);
- через порт ***outPreempted***, будучи вытесненной другой поступившей заявкой при заполненной очереди (если включен режим вытеснения);
- «вручную», путем вызова функции **remove()** или **removeFirst()**.

В первом случае объект **queue** покидает заявка, находящаяся в самом начале очереди (в нулевой позиции). Если заявка направлена в порт **outTimeout** или **outPreempted**, то она должна покинуть объект мгновенно. Если включена опция вытеснения, то объект **queue** всегда готов принять новую заявку, в противном случае при заполненной очереди заявка принята не будет.

Поступающие заявки помещаются в очередь в определенном порядке: либо согласно правилу FIFO (в порядке поступления в очередь), либо согласно приоритетам заявок. Приоритет может быть либо явно храниться в заявке, либо вычисляться согласно свойствам заявки и каким-то внешним условиям. Очередь с приоритетами всегда примет новую входящую заявку, вычислит её приоритет, и поместит в очередь в позицию, соответствующую её приоритету. Если очередь будет заполнена, то приход новой заявки вынудит последнюю хранящуюся в очереди заявку покинуть объект через порт **outPreempted**. *Но если приоритет новой заявки не будет превышать*

приоритет последней заявки, то тогда вместо неё будет вытеснена именно эта новая заявка.

Для выполнения условия постановки задачи воспользуемся последним способом вытеснения. Все запросы, вырабатываемые объектом **source**, имеют один и тот же приоритет. Поэтому при полном заполнении накопителя (5 запросов) теряться будет последний запрос. Уточните модель.

1. Выделите объект **queue**. На панели **Свойства** измените **Вместимость** с 15 на 5 запросов.

2. Здесь же установите **Разрешить вытеснение**.

3. Для уничтожения потерянных запросов вследствие полного заполнения накопителя нужно добавить второй объект **sink**. Откройте в **Палитре Библиотеку моделирования процессов** и перетащите блок **sink** на диаграмму (рис. 29). При перетаскивании объект пытается автоматически соединиться с входами имеющимися на диаграмме объектами.

4. Другой вариант - соедините порт **outPreempted** объекта **queue** с входным портом **InPort** блока **sink1**. Чтобы соединить порты, сделайте двойной щелчок мышью по одному порту, например, **outPreempted**, затем последовательно щёлкните в тех местах диаграммы, где вы хотите поместить точки изгиба соединителя.

5. После двойного щелчка по второму порту появится соединитель. Если выделить его мышью, то при правильном соединении портов конечные точки соединителя должны подсветиться зелеными точками. Если нет, то точки не были помещены точно внутрь портов, и их нужно будет туда передвинуть.

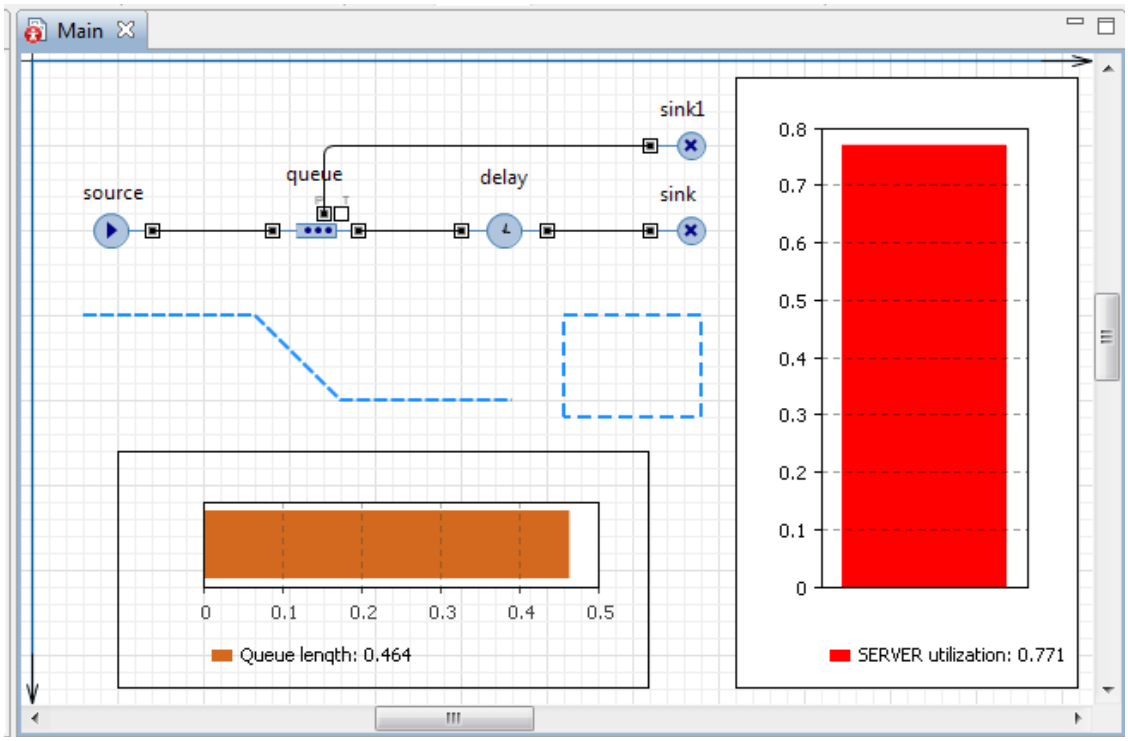

Рис. 29 - Уточненная модель

6. Запустите уточненную модель и понаблюдайте за ее работой. Сравните рис. 30 с рис. 22. На рис. 30 видно, что запросы при длине очереди в 5 запросов теряются, и ошибки при этом не возникает. Модель по ограничению ёмкости входного буфера и значениям других параметров соответствует постановке задачи.

Однако согласно постановке задачи требуется определить математическое ожидание времени обработки одного запроса и математическое ожидание вероятности обработки запросов.

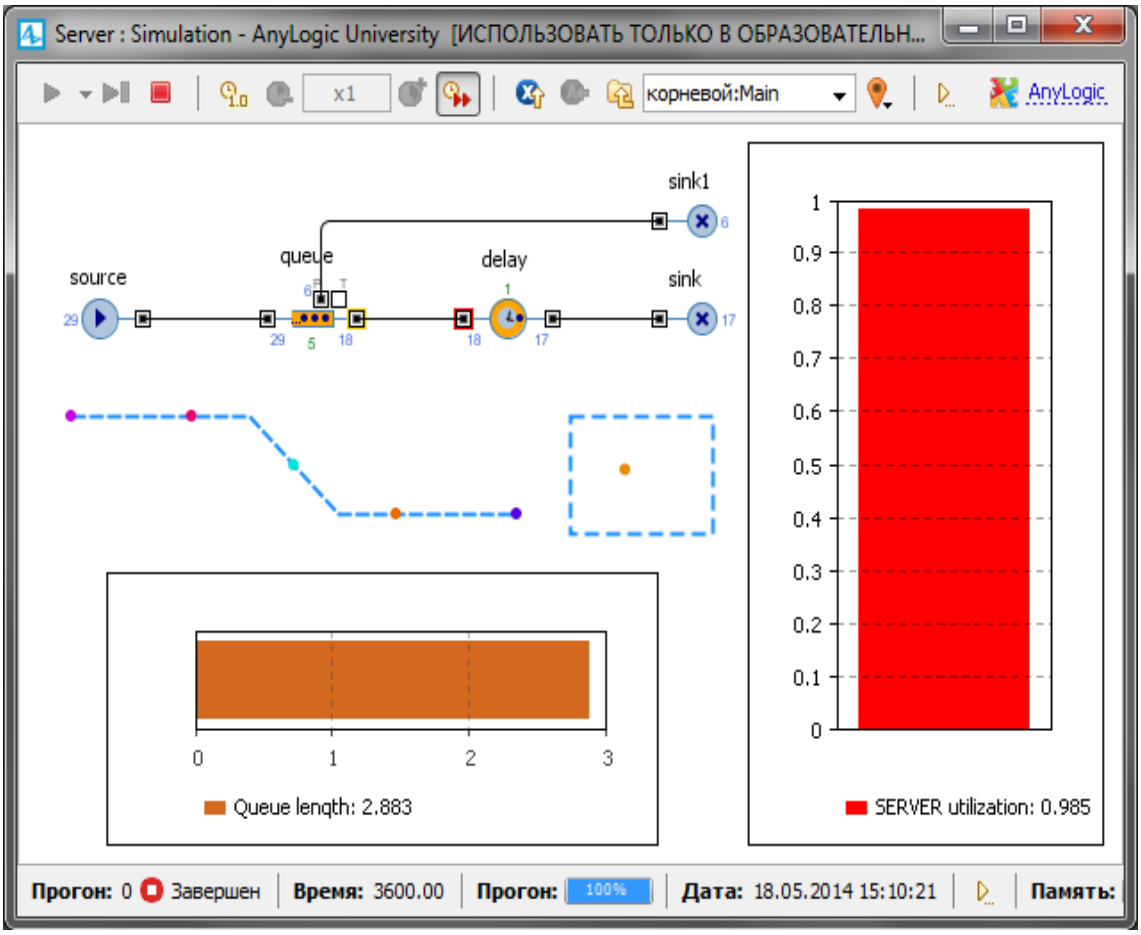

Рис. 30 - Работа модели согласно ёмкости входного буфера

Сбор статистики по показателям обработки запросов

Entity (заявка) являются базовым классом для всех заявок, которые создаются и работают с ресурсами в процессе, описанном вами с помощью диаграммы из объектов Библиотеки моделирования процессов. Entity по существу является обычным Java классом с теми функциональными возможностями, которые необходимы и достаточны для обработки и отображения анимации заявки объектами Библиотеки моделирования процессов. Эти функциональные возможности можно расширить добавлением дополнительных полей и методов и работой с ними из объектов диаграммы, описывающей моделируемый процесс.

Согласно постановке задачи нужно определять математическое ожидание времени и вероятности обработки запросов сервером.

Математическое ожидание или среднее время обработки одного запроса определяется как отношение суммарного времени обработки *n* запросов к их количеству, т. е. к *n*. Для определения суммарного времени нужно знать время обработки ***i***-го запроса. Для этого введем дополнительные поля:

- **time_vxod** — время входа запроса в буфер сервера;
- **time_vixod** — время выхода запроса с сервера (входа в блок **sink**).

Тогда **time_obrabotki=time_vixod-time_vxod**

Вероятность обработки запросов сервером определяется как отношение количества обработанных запросов к количеству всех поступивших запросов. Значит, нужно вести счет запросов на выходе источника запросов и на выходе с сервера (входе в блок **sink**). Для этого также введем дополнительные поля:

- **col_vxod**— количество поступивших всего запросов;
- **col_vixod**— количество обработанных сервером запросов.

Тогда **ver_obrabotki=col_vixod/col_vxod**

Замечание. Ничего необычного во введённых дополнительных полях нет. Это параметры реальных элементов потоков, в данном случае запросов. AnyLogic предоставляет возможность создавать запросы с теми параметрами, которые необходимы в модели.

Создание нестандартного Java класса

Для включения в запросы дополнительных полей необходимо создать нестандартный тип заявки. Это возможно двумя способами. Создадим первым способом тип заявок **Inquiry**.

1. В панели **Проект** щёлкните правой кнопкой мыши элемент модели верхнего уровня дерева и выберите из контекстного меню **Создать/Java класс**.

2. Появится диалоговое окно **Новый Java класс** (рис. 31). В поле **Имя** введите имя нового класса **Inquiry**.

3. В поле **Базовый класс** выберите из выпадающего списка **Entity** в качестве базового класса. Щёлкните кнопку **Далее**.

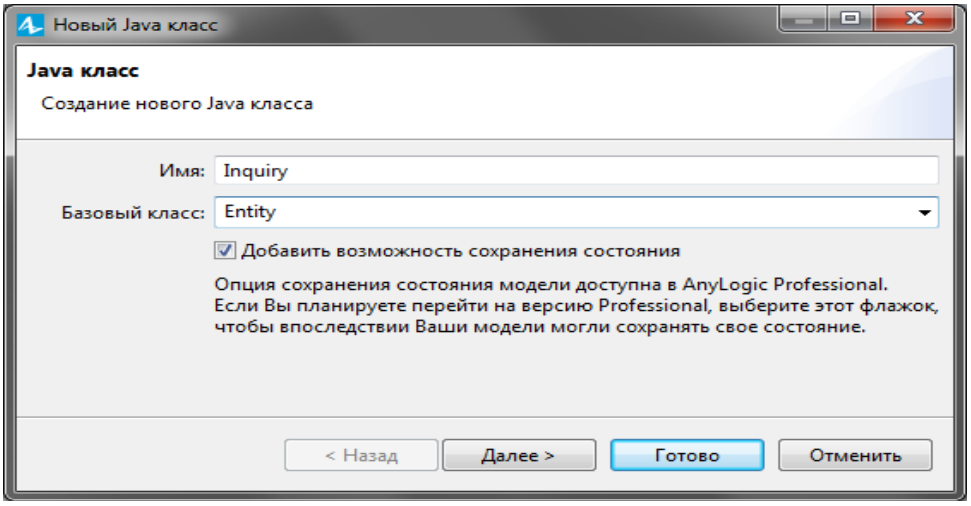

Рис. 31 - Диалоговое окно **Новый Java класс**

4. Появится вторая страница **Мастера создания Java класса**. (рис. 32).

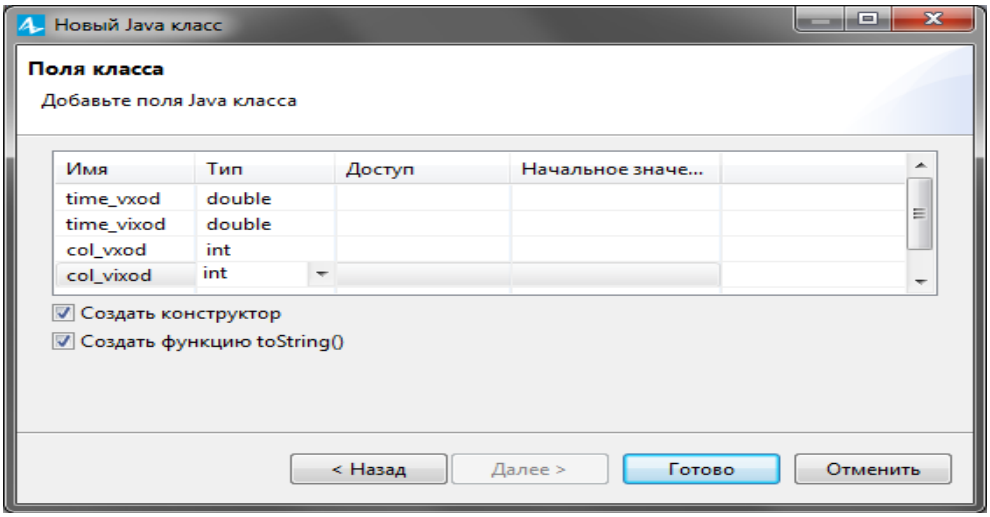

Рис. 32 - Вторая страница **Мастера создания Java класса**

5. Добавьте поля Java класса: **time_vxod** типа double, **time_vixod** типа double, **col_vxod** типа int, **col_vixod** типа int. Типы полей выбираются из выпадающего списка. Начальные значения всех параметров, поскольку не указаны, по умолчанию будут установлены равными нулю.

6. Оставьте выбранными флажки **Создать конструктор** и **Создать метод toString()**. Тогда у класса будут созданы сразу два конструктора: один, по умолчанию, без параметров, и второй, с параметрами, инициализирующими поля класса. Эти конструкторы используются объектами, создающими новые заявки, такие, как **Source**.

7. Щёлкните кнопку **Готово**. Вы увидите редактор кода, в котором будет показан автоматически созданный код вашего Java класса (рис. 33). Закройте редактор, щелкнув крестик в закладке рядом с его названием.

8. Теперь нужно преобразовать Java класс в тип агента. Для этого щёлкните правой кнопкой мыши в панели **Проект** только что созданный Java класс и в контекстном меню выберите **Преобразовать Java класс в тип агента**.

9. Появится окно с автоматически созданными параметрами нестандартного типа заявок **Inquiry** (см. рис. 38).

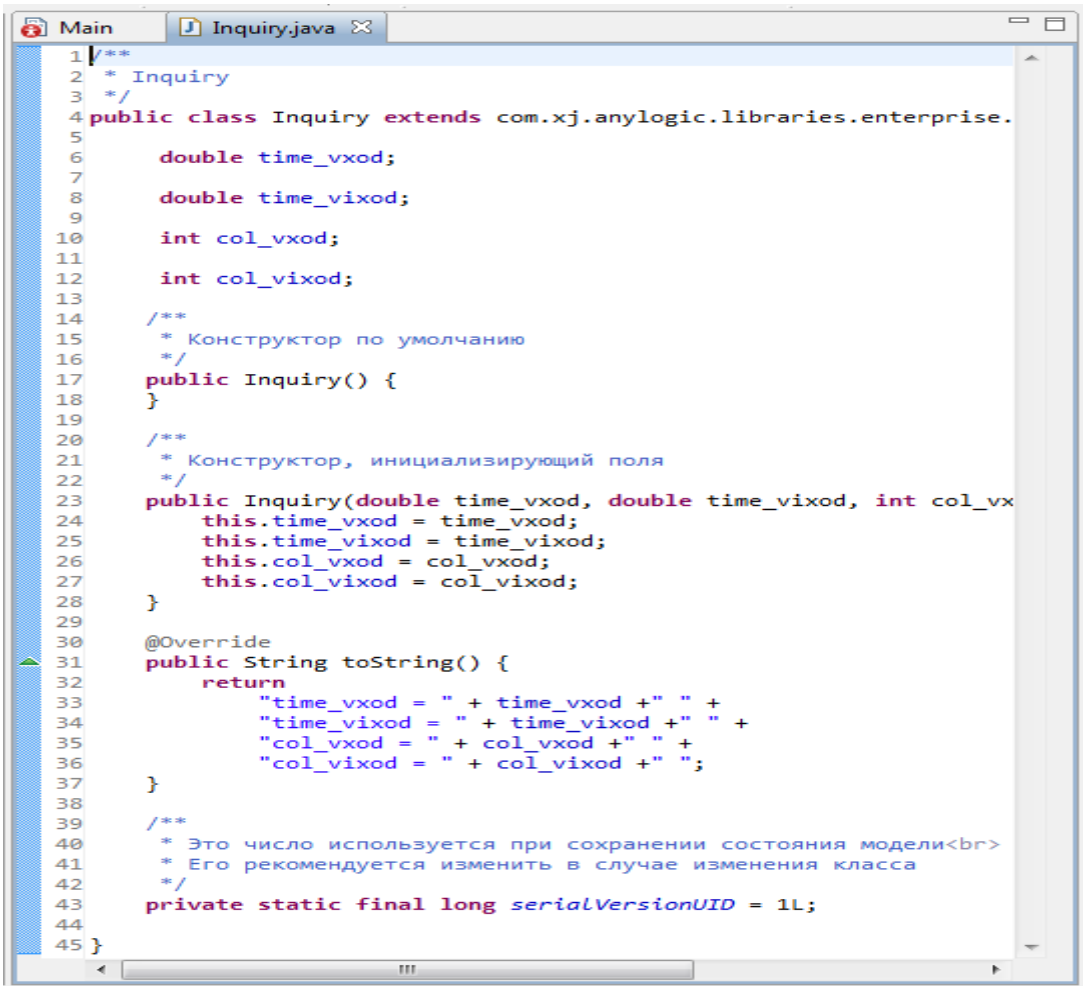

Рис. 33 - Окно редактора кода созданного нестандартного класса

10. Закройте оно, щелкнув крестик в закладке рядом с его названием. Удалите всё, что касается только что созданного Java класса и заявки. Перейдём к созданию непосредственно нестандартного типа заявки вторым способом.

Создайте тип заявок **Inquiry**.

1. Откройте палитру **Библиотека моделирования процесов**.

2. Перетащите элемент **Тип заявки** в графический редактор. Появится тип заявки **Entity** (рис. 34).

3. Появится диалоговое окно **Создание агента** (рис. 35).

Шаг 1. Анимация агента.

В поле **Имя нового агента:** введите **Inquiry**.

Выберите анимацию агента: установите 2D и выберите из выпадающего списка, например, **Сообщение**.

Щёлкните **Далее**.

4. Появится диалоговое окно **Создание агента. Шаг 2. Параметры агента** (рис. 36).

5. Щёлкните **<добавить…>**. В поле **Параметр:** введите **time_vxod** (рис. 37).

6. Из выпадающего списка **Тип:** выберите **double**.

7. Щёлкните второй раз **<добавить…>**. В поле **Параметр:** введите **time_vixod**.

8. Из выпадающего списка **Тип:** выберите **double**.

9. Щёлкните третий раз **<добавить…>**. В поле **Параметр:** введите **col_vxod**.

10. Из выпадающего списка **Тип:** оставьте **int**.

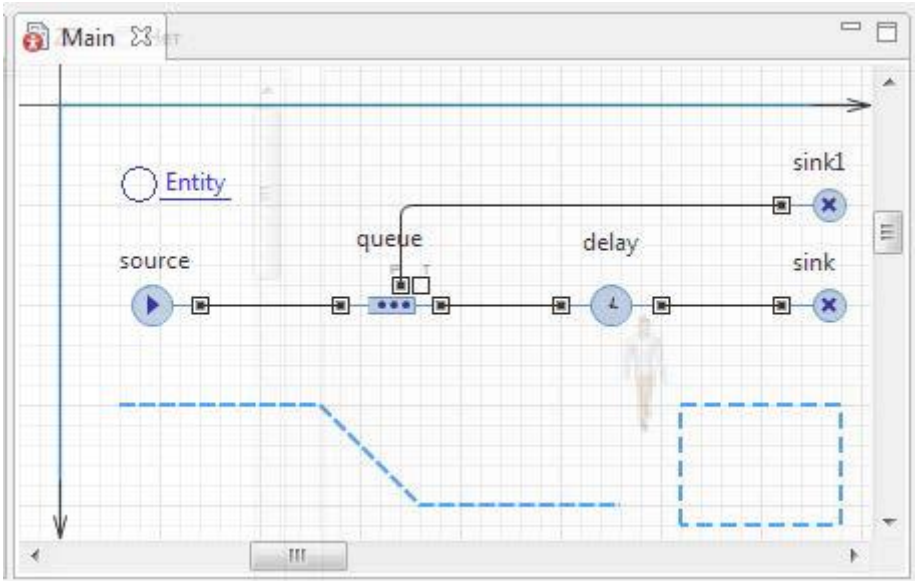

Рис. 34 - Появился тип заявки Entity

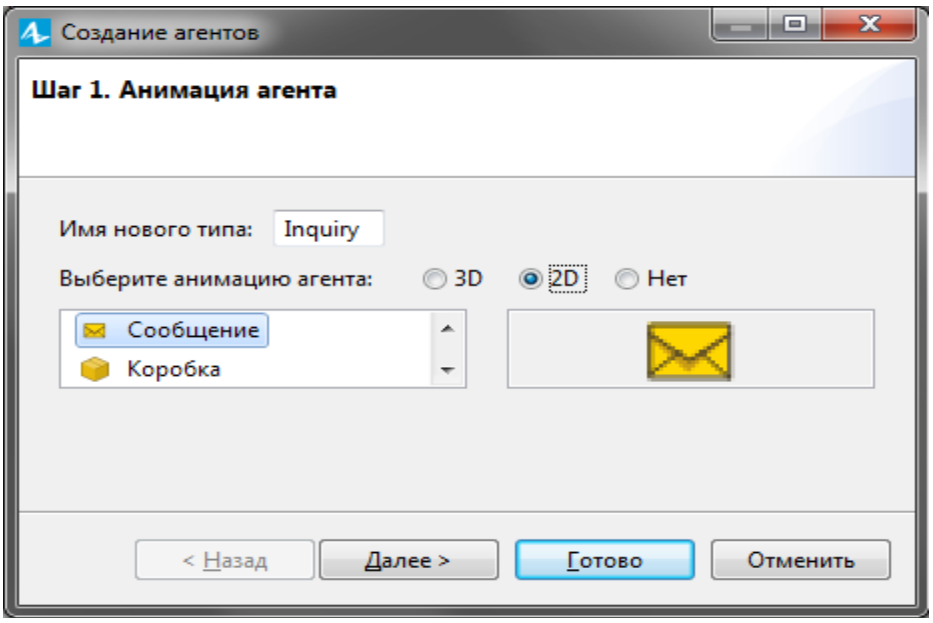

Рис. 35 - Диалоговое окно **Создание агента. Шаг 1. Анимация агента**

1. Щёлкните третий раз **<добавить…>**. В поле **Параметр:** введите **col_vixod**.
2. Из выпадающего списка **Тип:** оставьте **int**.

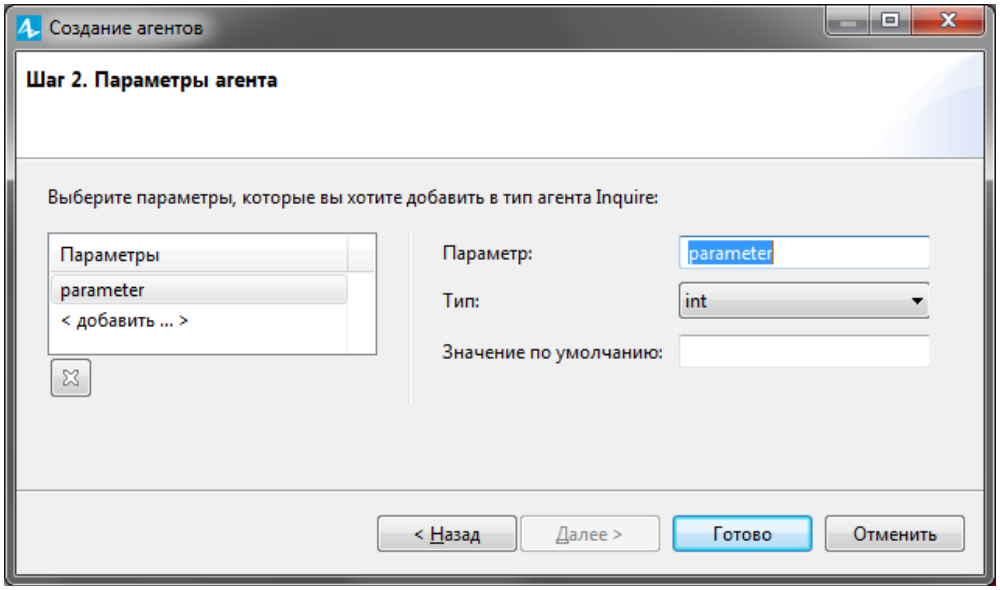

Рис. 36 - Диалоговое окно **Создание агента. Шаг 2. Параметры агента**

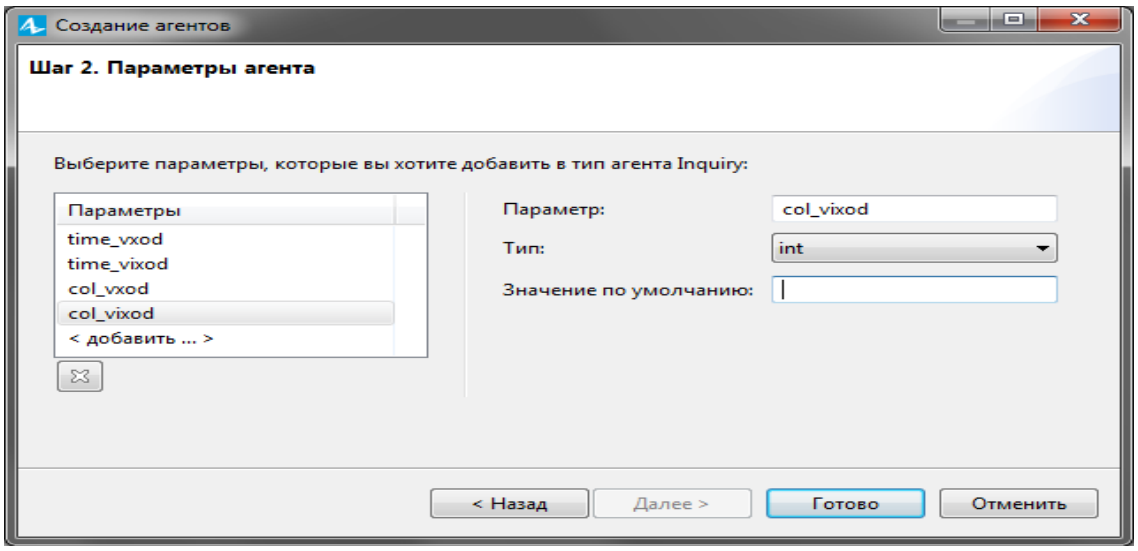

Рис. 37 - Диалоговое окно **Создание агента. Шаг 2. Параметры агента** с установленными параметрами нестандартного типа заявок **Inquiry**

3. Так как в поле **Значение по умолчанию** мы не устанавливали никаких значений, то всем параметрам будет установлен 0.

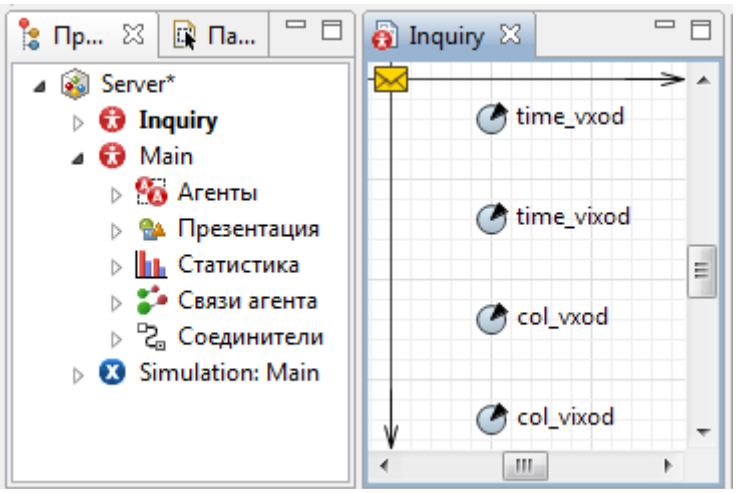

Рис. 38 - Окно с параметрами нестандартного типа заявки Inquiry

4. Щёлкните кнопку **Готово**. Вы увидите окно, в котором будут показаны автоматически созданные параметры нестандартного типа заявок **Inquiry** (рис. 38). Закройте оно, щелкнув крестик в закладке рядом с его названием.

Добавление элементов статистики

Для сбора статистических данных о времени обработки запросов сервером необходимо добавить элемент статистики. Этот элемент будет запоминать значения времен для каждого запроса. На основе этого он предоставит пользователю стандартную статистическую информацию (среднее, минимальное, максимальное из измеренных значений, среднеквадратичное отклонение и т.д.).

1. Чтобы добавить элемент сбора данных гистограммы на диаграмму, перетащите элемент **Данные гистограммы** с палитры **Статистика** на диаграмму активного класса.

2. Задайте свойства элемента (рис. 39):

- измените **Имя:** на time_obrabotki;
- сделайте **Кол-во интервалов:** равным 50;
- задайте **Нач. размер интервала:** 0.01.

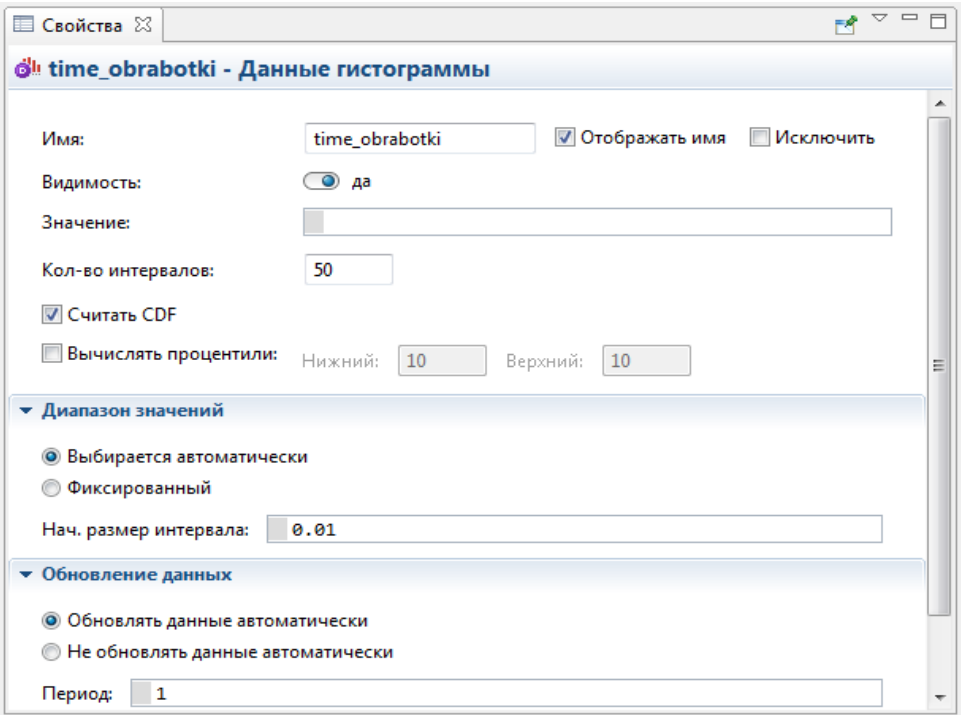

Рис. 39 - Элемент сбора статистики о времени обработки запросов

Добавьте еще элемент сбора статистики для определения вероятности обработки запросов.

1. Перетащите элемент **Данные гистограммы** с палитры **Статистика** на диаграмму активного класса.

2. Задайте свойства элемента (рис. 40):

- измените **Имя:** на **ver_obrabotki**;
- сделайте **Кол-во интервалов:** равным 50;
- задайте **Нач. размер интервала:** 0.01.

Диаграмма после добавления элементов сбора статистики представлена на рис. 41.

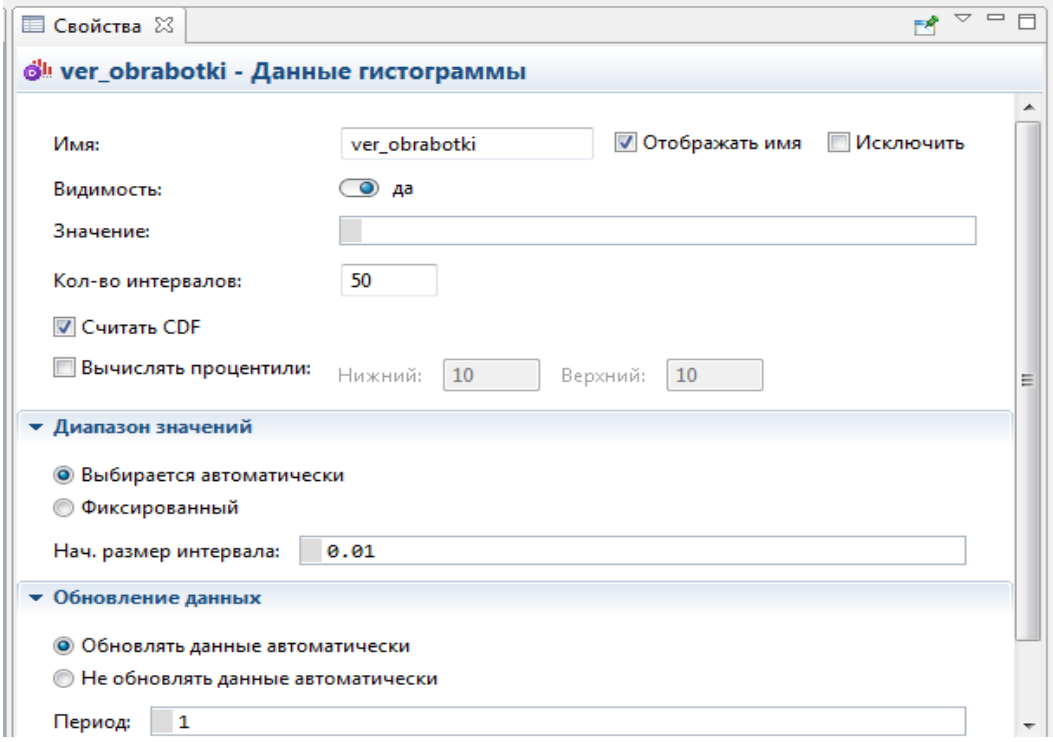

Рис. 40 - Элемент сбора статистики о вероятности обработки запросов

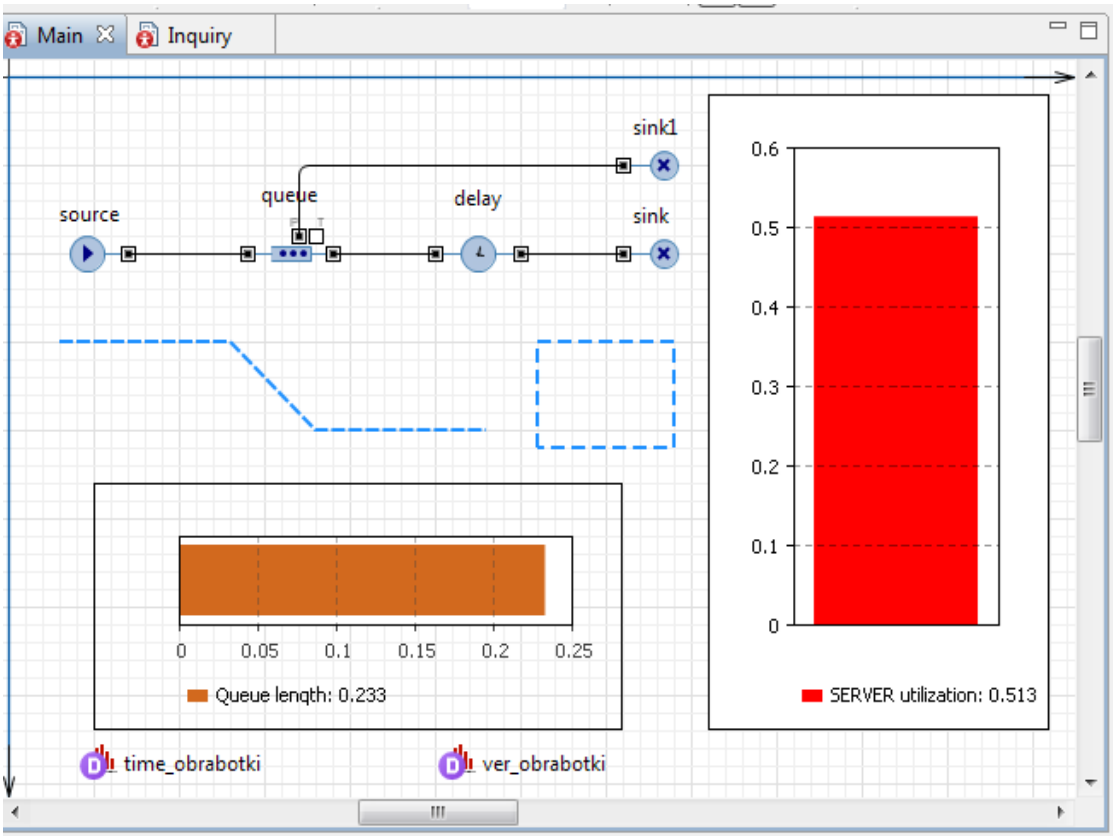

Рис. 41 - Диаграмма после добавления элементов сбора статистики

Изменение свойств объектов диаграммы

Чтобы создавать заявки нестандартного типа, как в случае **Inquiry**, нужно поместить вызов конструктора этого типа в поле **Новая заявка** объекта **source**. Но, несмотря на то, что заявки в потоке теперь и будут типа **Inquiry**, остальные объекты диаграммы будут продолжать их считать заявками типа **Entity**.

Поэтому они не позволят явно обращаться к дополнительным полям класса **Inquiry**. Чтобы разрешить доступ к полям нестандартного типа заявки в коде динамических параметров объектов потоковой диаграммы, нужно указать имя нестандартного типа заявки в качестве **Типа заявки** этого объекта. В потоковой диаграмме с учётом блока **source** всего пять объектов.

Измените их свойства.

1. Измените свойства объекта **source** (рис. 42):

- введите **Inquiry** в поле **Тип заявки:**. Это позволит напрямую обращаться к полям типа заявки **Inquiry** в коде динамических параметров этого объекта;
- выберите из выпадающего списка **Inquiry()** в поле **Новая заявка:**. Теперь этот объект будет создавать заявки нашего типа **Inquiry**;
- введите **entity.time_vxod=time();** в поле **Действия При выходе:**. Код будет сохранять время создания заявки-запроса в параметре **time_vxod** типа заявки **Inquiry**.

Функция **time()** возвращает текущее значение модельного времени.

2. Измените свойства объекта **queue**: введите **Inquiry** в поле **Тип заявки:**.

3. Измените свойства объекта **delay**: введите **Inquiry** в поле **Тип заявки:**.

4. Измените свойства объекта **sink1**: введите **Inquiry** в поле **Тип заявки:**.

5. Измените свойства объекта **sink**: введите **Inquiry** в поле **Тип заявки:**;

введите в поле **Действие при входе** следующие коды:

time_obrabotki.add(time()-entity.time_vxod);

Этот код добавляет время обработки одного запроса в объект сбора данных гистограммы **time_obrabotki**. Данное время определяется как разность между текущим модельным временем **time()** и временем входа запроса в модель**. add** — встроенная функция добавления элемента в массив.

entity.col_vixod=sink.count();

entity.col_vxod=source.count();

Эти коды заносят количество запросов, вошедших в блок **sink** и вышедших из блока **source** соответственно. **count()**— встроенная функция этих блоков, возвращает количество вошедших в блок **sink** и количество вышедших из блока **source** заявок.

ver_obrabotki.add(entity.col_vixod/entity.col_vxod);

Этот код добавляет относительную долю обработанных запросов в объект сбора данных гистограммы **ver_obrabotki** при поступлении каждого обработанного запроса в блок **sink**. На основе множества таких относительных долей определяется математическое ожидание вероятности обработки запросов сервером.

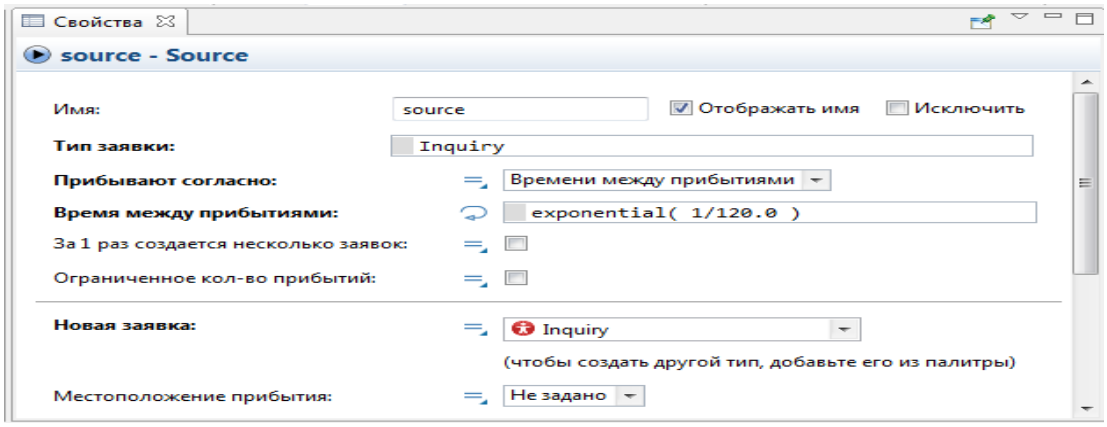

Рис. 42 - Объект **source** с изменёнными свойствами

6. Запустите модель. Появится сообщение об ошибке. Щёлкните **Продолжить**. Появится второе сообщение (рис. 43).

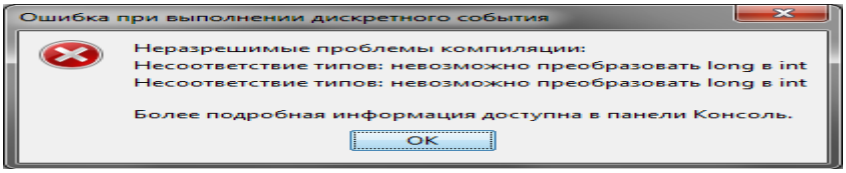

Рис. 43 - Второе сообщение об ошибке

7. Щёлкните выделенный Java код в панели **Консоль** Main.java.399 (рис. 44). Появится код с выделенными ошибками.

Был установлен тип **int** для **col_vxod** и **col_vixod** (см. рис. 32). Изменим этот тип на **double**.

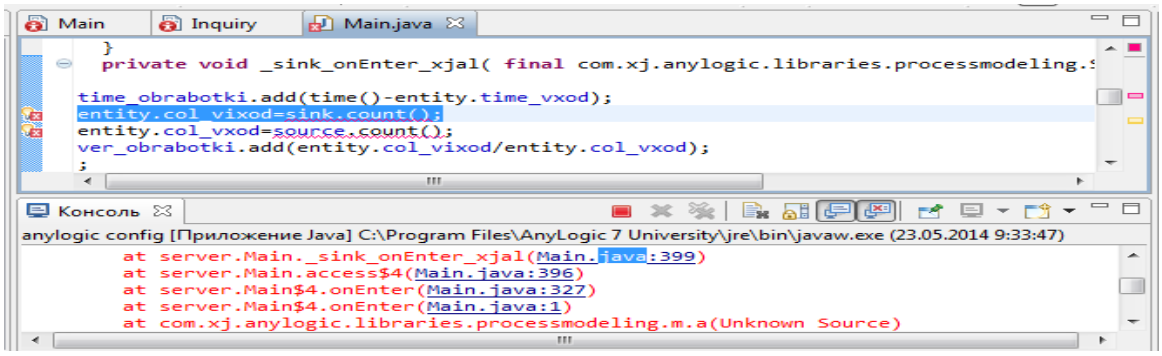

Рис. 44 - Информация об ошибке в панели **Консоль**

Удаление и добавление новых полей типа заявок

Вместо использования поля **time_vixod** была использована функция **time()**, возвращающая текущее значение модельного времени.

Удалите поле **time_vixod**.

1. В поле **Проект** дважды Щёлкните кнопку **Inquiry**. Откроется окно **Inquiry** (см. рис. 38).

2. Удалите **time_vixod**, выделив его и нажав Delete.

3. Выделите **col_vxod**. Из выпадающего списка **Тип:** вместо **int** выберите **double** (рис. 45).

4. Выделите **col_vixod**. Из выпадающего списка **Тип:** вместо **int** выберите **double**.

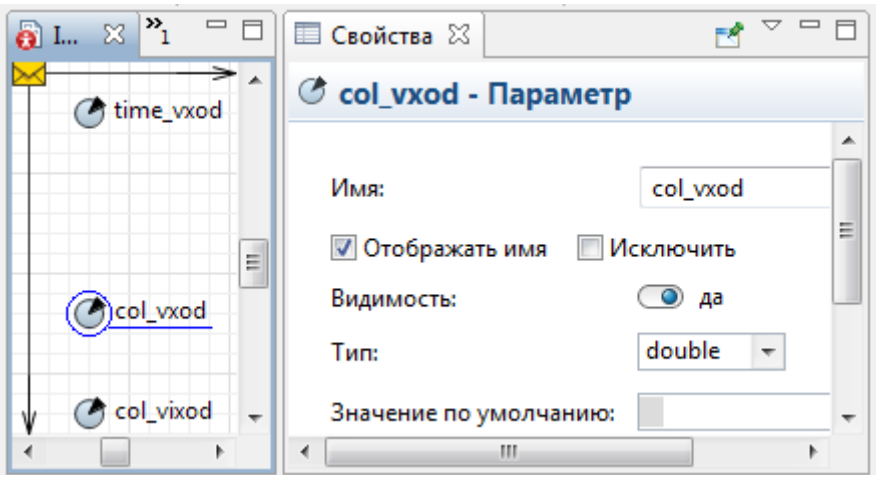

Рис. 45 - Окно после удаления поля time_vixod

5. Из процедуры удаления поля следует, что так же можно вводить новые дополнительные поля нестандартного типа заявок. Например, перетащите из библиотеки **Основная** элемент **Параметр**. Дайте ему любое имя и установите **Тип:** из выпадающего списка. В дальнейшем это поле нестандартного типа заявки можно использовать в кодах модели.

Итак, все условия постановки задачи выполнены. Чтобы наблюдать за работой модели, установите, что время остановки модели не задано. Запустите модель. (рис. 46).

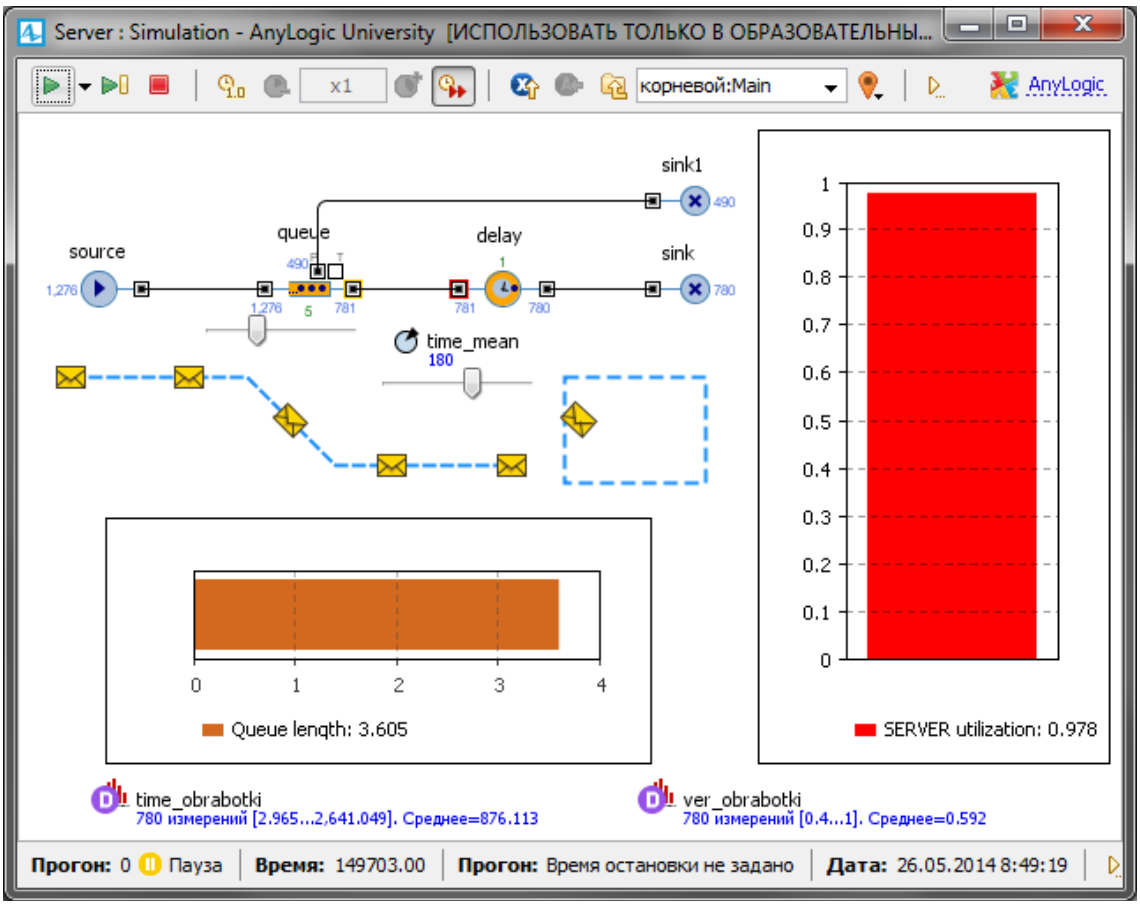

Рис. 46 -Фрагмент работы модели

Добавление параметров и элементов управления

Активный объект может иметь параметры. Параметры обычно используются для задания статических характеристик объекта. Но значения параметров при необходимости можно изменять во время работы модели. Для этого нужно написать код обработчика события, то есть действий, которые должны выполняться при изменении значения параметра.

Создайте параметр **time_mean** объекта **delay**.

1. В **Палитре** выделите **Основная**.

2. Перетащите элемент **Параметр** на диаграмму класса **Main** и разместите ниже объекта **delay**, чтобы было видно, к какому объекту относится параметр.

3. Перейдите на панель **Свойства** (рис. 47).

4. В поле **Имя** введите имя параметра **time_mean** (среднее время). По этому имени параметр будет доступен из кода.

5. Задайте тип параметра **double**.

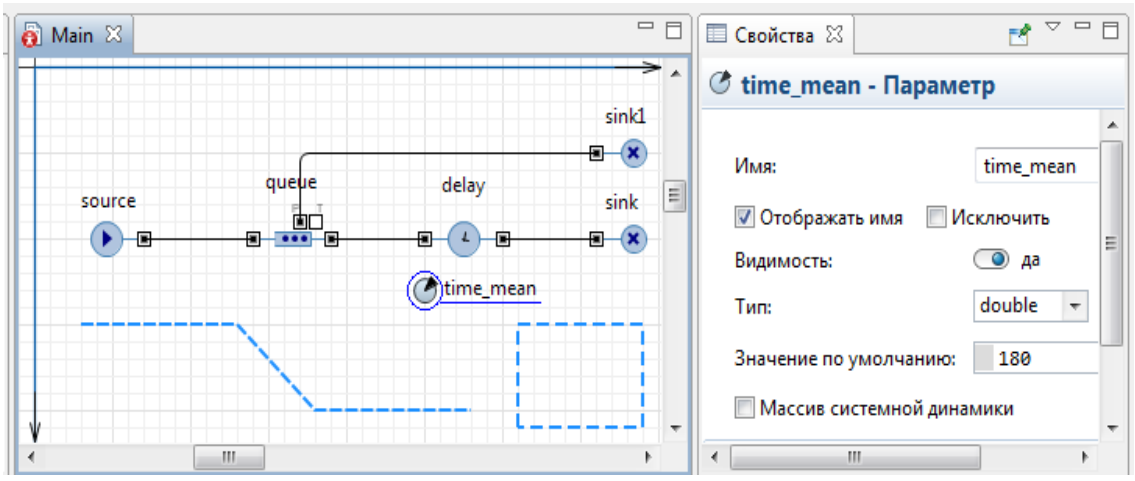

Рис. 47 - Окно установки свойств элемента **Параметр**

6. В поле **Значение по умолчанию** установите 180. Если значение не задано явно, по правилам Java оно будет равно нулю.

7. Выделите объект **delay**.

8. На панели **Свойства** в поле **Время задержки** вместо выражения ***exponential(1/180.0)*** введите выражение ***exponential(1/time_mean).***

1. Откройте палитру **Элементы управления** и перетащите элемент **Бегунок** из палитры на диаграмму класса **Main** (рис. 48).

2. Поместите бегунок под параметром **time_mean**, чтобы было понятно, что с помощью этого бегунка будет меняться среднее время обработки запросов объектом **delay**.

3. Пусть требуется варьировать среднее время от 1 до 300. Поэтому введите 1 в поле **Минимальное значение:**, а 300— в поле **Максимальное значение:** (рис. 49).

4. Установите флажок **Связать с:** и в активизированное поле введите time_mean.

Пусть теперь вы хотите также изменять ёмкость буфера в ходе моделирования. Используйте для этого также бегунок.

1. Откройте палитру **Элементы управления** и перетащите элемент **Бегунок** из палитры на диаграмму класса Main (рис. 1.49).

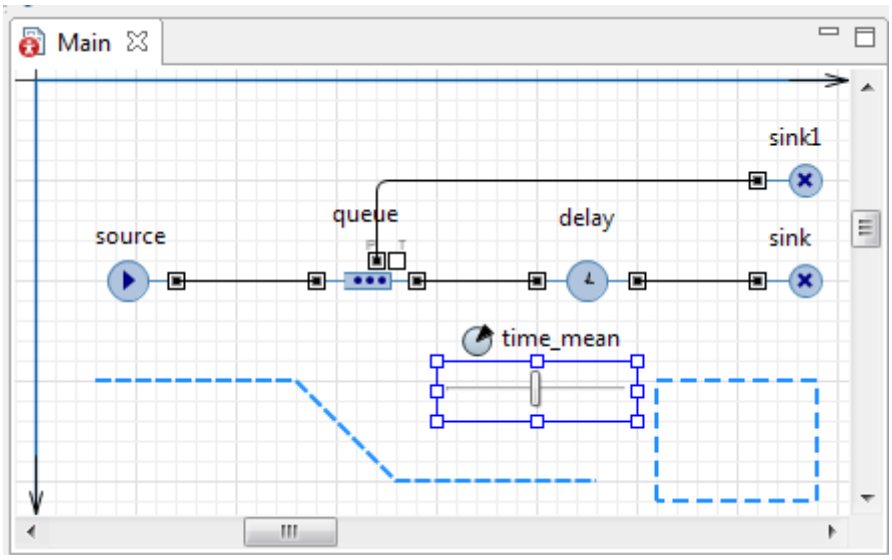

Рис. 48 - Установка элемента управления **Бегунок**

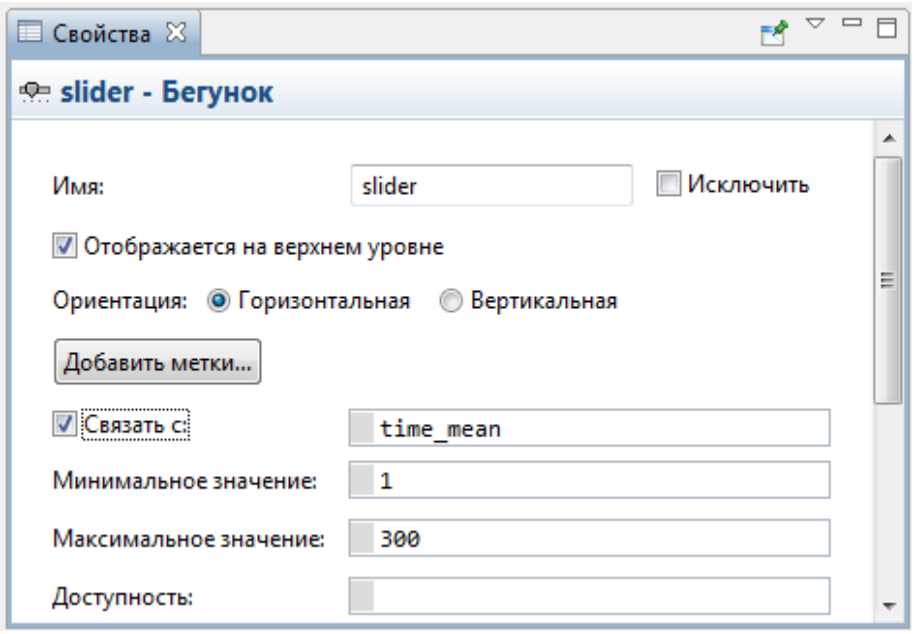

Рис. 49 - Окно установки свойств элемента управления **Бегунок**

2. Поместите бегунок под объектом **queue**, чтобы было понятно, что с помощью этого бегунка будет меняться вместимость данного объекта, имитирующего входной буфер.

3. Пусть требуется варьировать ёмкость буфера от 0 до 15 запросов. Поэтому введите 15 в поле **Максимальное значение**.

4. Установите флажок **Связать с:** и в активизированное поле введите **queue.capacity**.

5. Запустите модель. Теперь можно изменять в процессе моделирования ёмкость входного буфера и среднее время обработки запросов с помощью бегунков. Командой **Приостановить можно** приостановить работу модели, изменить значения параметров, а затем продолжить моделирование (рис. 50).

6. Остановите модель и перейдите на диаграмму класса **Main**.

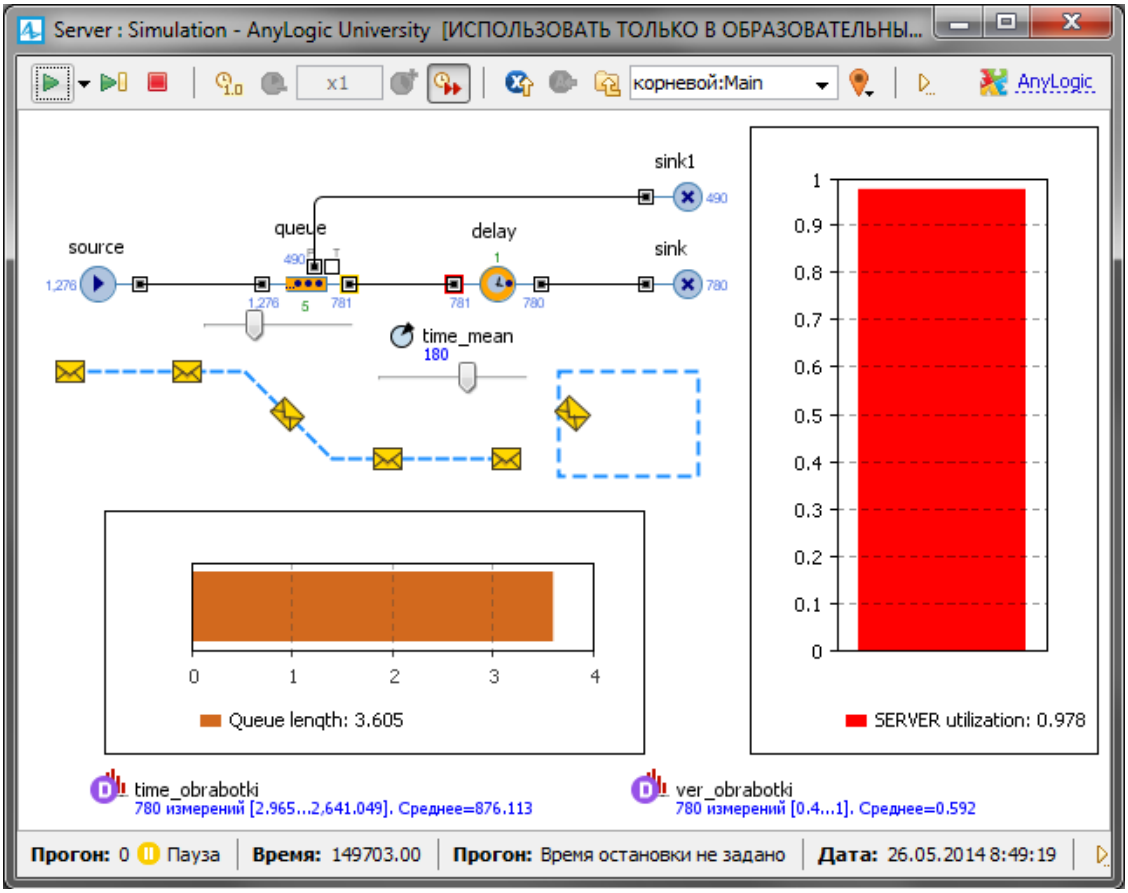

Рис. 50 - Фрагмент работы модели с добавленным элементом **Параметр** и элементами управления

Кроме бегунка существует и другой способ изменения свойств объектов во время выполнения модели: нужно щёлкнуть по элементу, войти в режим редактирования и ввести новое значение в одной из закладок всплывающего окна инспекта. Поэтому заранее не нужно продумывать, значения каких параметров планируется изменять, и не добавлять специальные элементы управления (например, бегунки).

Изменение значения в окне инспекта поддерживается для следующих элементов: простая переменная; параметр; накопитель.

И для следующих типов: численные; логический (boolean); текстовый (String).

1. Удалите элемент **Бегунок** для **Параметра** time_mean.
2. Запустите модель и приостановите её.
3. Щёлкните по значку **Параметра** time_mean.
4. Перейдите в режим редактирования (рис. 51).
5. Введите новое значение: 240.0.
6. Закройте инспект. Рядом с элементом **Параметр** появится введённое значение 240.0.
7. Запустите модель с новым свойством объекта **delay**.

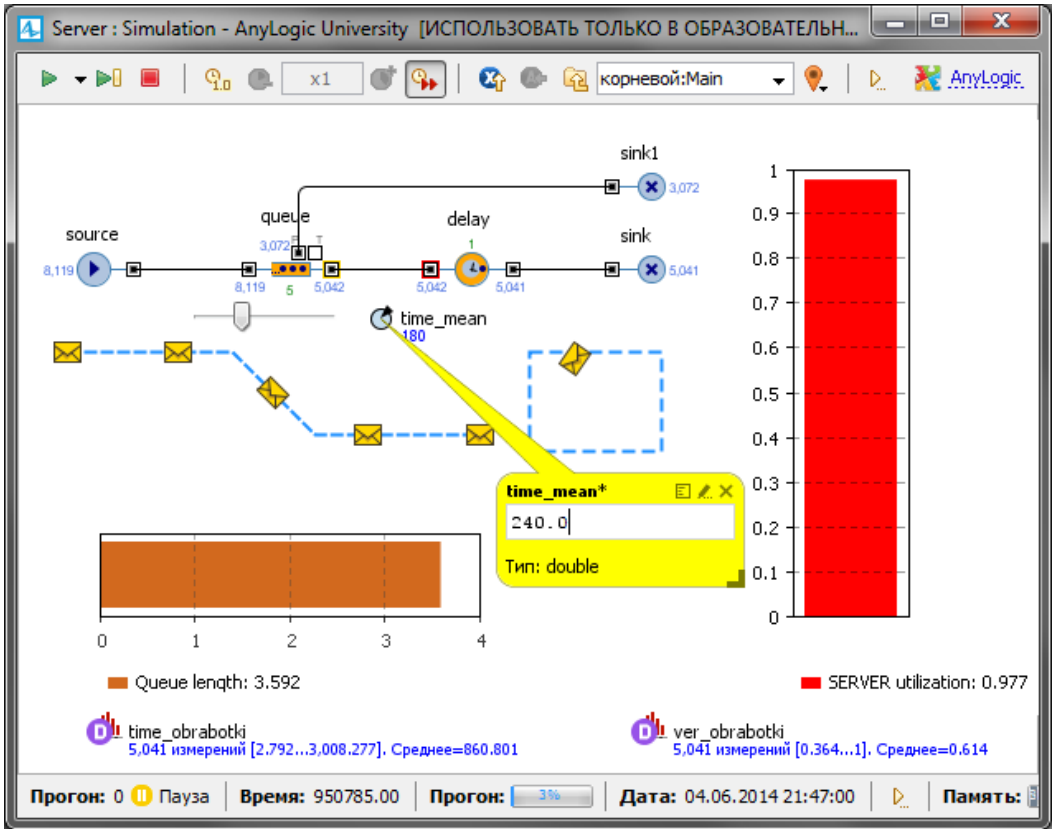

Рис. 51 - Ввод нового значения параметра в окно инспекта

Добавление гистограмм

1. Перетащите элемент **Гистограмма** из палитры **Статистика** в то место графического редактора, куда нужно её поместить.

2. Укажите, какой элемент сбора данных хранит данные, которые нужно отображать на гистограмме: щёлкните кнопку **Добавить данные** и введите в поле **Данные** имя соответствующего элемента: **time_obrabotki** (рис. 52). Установите **Отображать среднее**.

3. В поле **Заголовок:** введите **Histogram Time obrabotki**.

Добавим на диаграмму нашего потока гистограмму, которая будет отображать собранную вероятностную статистику.

1. Перетащите элемент **Гистограмма** из палитры **Статистика**.

2. Щёлкните кнопку **Добавить данные** и введите в поле **Данные** имя элемента: **ver_obrabotki**. Установите **Отображать среднее**.

3. В поле **Заголовок:** введите **Histogram Ver obrabotki**.

4. Запустите модель. Фрагмент работы показан на рис. 53.

Замечание. Обратите внимание, что после нового запуска модели **time_mean**=180, хотя ранее мы изменили его значение на 240.

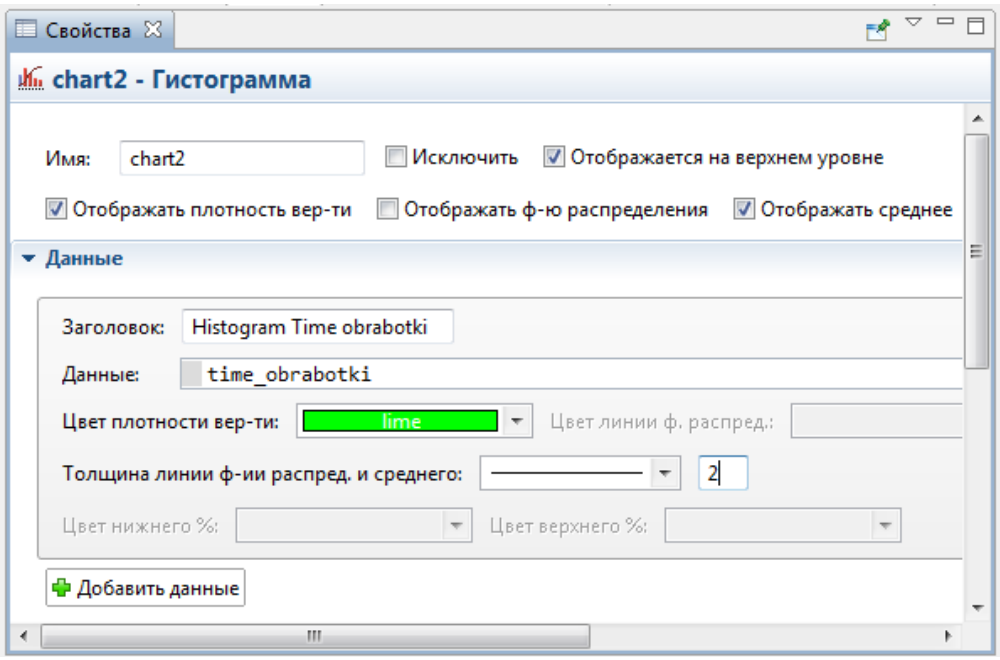

Рис. 52 - Окно установки свойств элемента **Гистограмма**

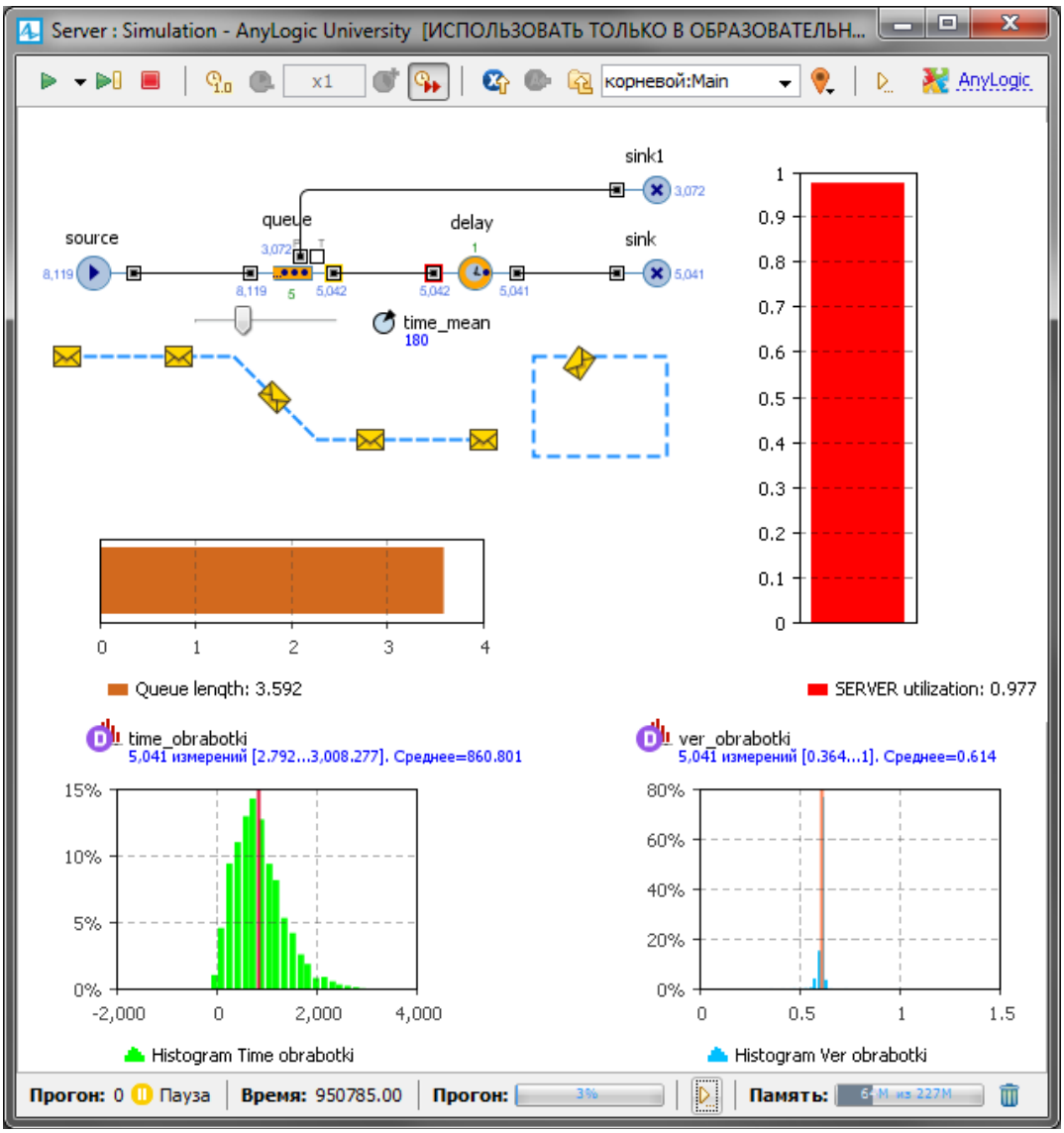

Рис. 53 - Фрагмент работы модели с элементом управления и гистограммами

Изменение времени обработки запросов сервером

Построенная модель соответствует постановке задачи. В ней, с целью упрощения процесса построения первой модели, время обработки запросов сервером было принято распределённым по показательному (экспоненциальному) закону со средним значением *T2*=3мин.

Однако в модели время обработки поступающих запросов зависит от производительности сервера *Q* =·5610оп/с и вычислительной сложности запросов, распределенной по нормальному закону с математическим ожиданием $S1=6\cdot10^7$ оп и среднеквадратическим отклонением $S2= 2\cdot10^5$ оп.

Кроме того, в модели определяется среднее количество запросов, обработанных за время моделирования 3600 с.

Внесите в модель изменения для аналогичного расчёта времени обработки запросов.

1. Удалите элемент **Параметр** с именем **time_mean** элемент **Бегунок** для элемента **queue**.

2. Из палитры **Основная** перетащите три элемента **Параметр** на диаграмму класса Main (рис. 54).

3. В поле **Имя** каждого из элементов введите S1_, S2_ и Q_ соответственно. Выберите **Тип** double.

4. В поле **Значение по умолчанию** каждого из элементов введите 60000000, 200000 и 600000 соответственно.

5. Перетащите элемент **Переменная**.

6. В поле **Имя** укажите **KolZap**.

7. Выделите объект **delay**.

8. В поле **Время задержки** вместо ***exponential(1/time_mean)*** введите: **(normal(S2_,S1_))/Q_**

9. Выделите объект **sink**. В поле **Действие при входе** к имеющемуся там коду добавьте код:

KolZap=sink.in.count() / 9604.0;

Для получения результатов моделирования с доверительной вероятностью $\alpha = 0,95$ и точностью $\varepsilon = 0,01$ нужно выполнить 9604 прогонов модели:

$$N = t_\alpha^2 \frac{p(1-p)}{\varepsilon^2} \approx 1,96^2 \frac{0,5^2}{0,01^2} \approx 9604,$$

где t_α= 1,96 — табулированный аргумент функции Лапласа, *p*− ожидаемая вероятность исхода события, в данном случае вероятность обработки запросов сервером.

Расчёт проведен для так называемого «худшего» случая, то есть в предположении, что ожидаемая вероятность обработки запросов *p*=0,5.

Увеличим время моделирования в AnyLogic-модели в 9604 раз. А так как статистические данные о количестве обработанных запросов собираются за всё время моделирования, увеличенное в 9604 раз, то для получения

среднего значения это количество нужно разделить на 9604, что и предусмотрено в коде.

Рис. 54 - Элементы AnyLogic-модели, соответствующие постановке задачи

10. Показатели моделируемой системы нужно определить в течение 3600 с, поэтому время моделирования в AnyLogic составит 3600*9604 = 34574400 единиц модельного времени.

11. В панели **Проект** выделите **Simulation**. На странице **Модельное время** в поле **Установить** выберите **В заданное время**.

12. В поле **Конечное время** установите 34574400.

13. Запустите модель и дождитесь окончания моделирования.

Результаты моделирования приведены на рис. 55.

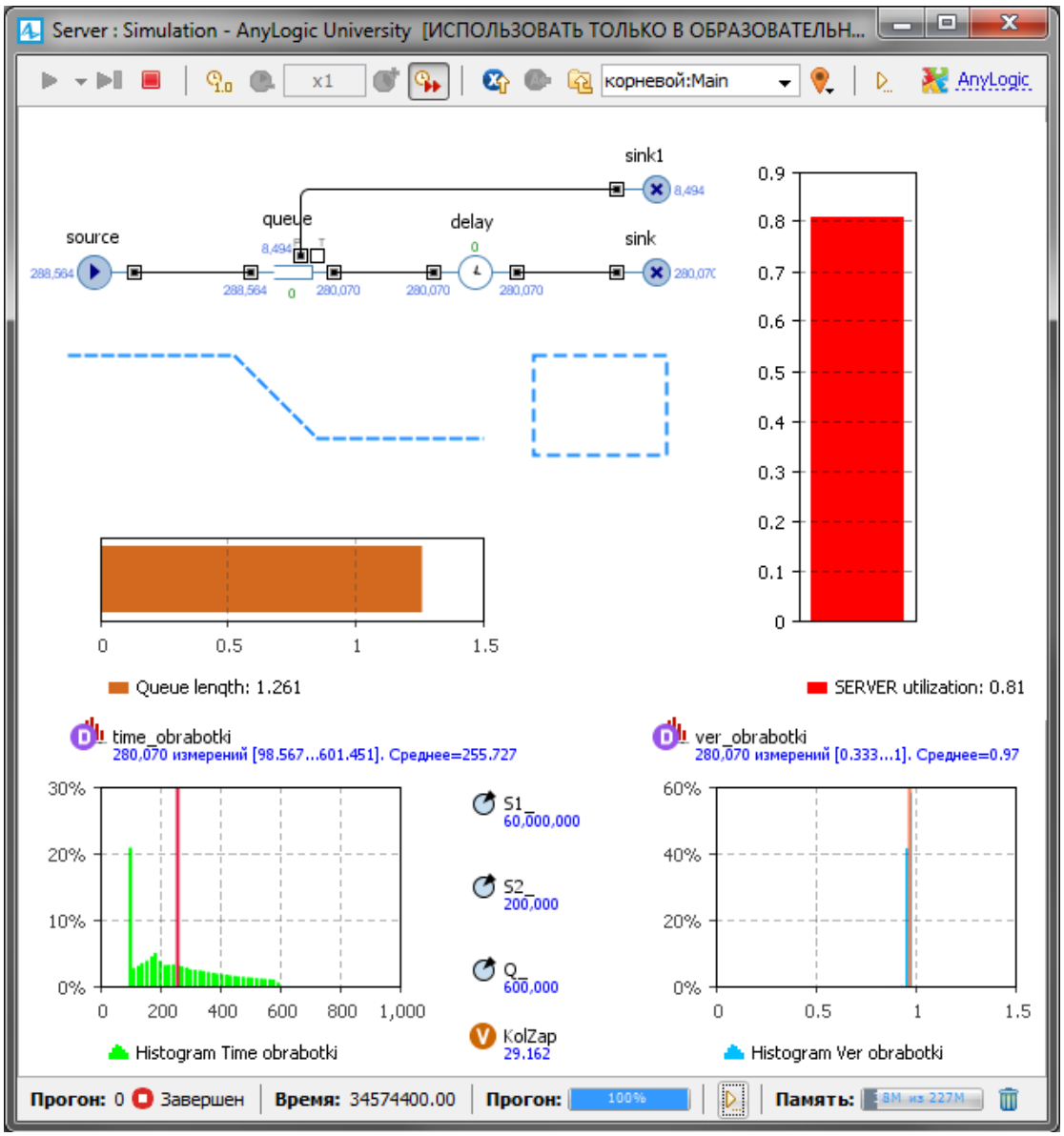

Рис. 55 - Результаты моделирования обработки запросов сервером